Daily Organizer -- Complete Project Documentation

Table of Contents

1. Project Overview
2. Technology Stack
3. Architecture Overview
4. Project Structure
5. Backend Deep Dive
6. Frontend Deep Dive
7. Data Models
8. Security
9. Theme System
10. State Management
11. Key Design Decisions
12. Setup & Running Guide
13. Google Calendar Integration
14. Auto Status Updates
15. Future Scope & Enhancements
16. Mobile Responsive Design
17. Loading Screen for Cold Start
18. Clickable Dashboard Stat Cards
19. Production Deployment
20. Form Reset Fix

---

1. Project Overview

What the App Does

Daily Organizer is a full-stack personal productivity application designed for individuals who need a single, unified place to manage every aspect of their daily life. Unlike traditional to-do list applications that only handle tasks, Daily Organizer treats daily planning as a multi-faceted activity. A typical day involves not just tasks to complete, but also trips to take, trains to catch, dinners to attend, meetings to join, events to remember, and reminders to act on. Daily Organizer brings all of these plan types together into one cohesive interface.

Who It's For

The application is built for anyone who wants to go beyond simple task management. Whether you are a professional juggling meetings and deadlines, a student tracking study goals and travel schedules, or someone who simply wants to organize their daily routines, Daily Organizer provides the right level of structure without being overwhelming. The app's three-pillar approach ensures that you are not just tracking what needs to get done today, but also making progress on long-term goals and keeping track of things you want to buy.

The Problem It Solves

Most productivity tools force users to shoehorn every type of activity into a "task." A train departure is not a task. A dinner reservation is not a task. Yet these are things that populate our daily schedules and need to be tracked alongside actual work items. Daily Organizer solves this by introducing a multi-type plan system with seven distinct plan types, each with its own icon, color, and contextual fields (like location for trips and dinners, or start/end times for trains and meetings).

Three Pillars

The application is organized around three core pillars:

1. **Daily Plans** -- The heart of the app. Create, organize, filter, and track plans across seven types (task, trip, train, dinner, meeting, event, reminder). Each plan supports scheduling with due dates, time ranges, locations, priority levels, status tracking, and

automatic status transitions based on scheduled times. The form dynamically adapts to show context-specific fields for each plan type (e.g., boarding station for trains, meeting link for meetings).

2. **Goal Tracking** -- A three-level hierarchy (Goal > Milestone > Mini-Goal) that lets users break down ambitious objectives into manageable steps. Progress is automatically calculated as milestones are completed, giving a clear visual indicator of how close you are to achieving each goal.

3. **Buy List** -- A wishlist system for tracking products the user wants to purchase. Each item can have a price, currency, product URL, image, and priority. Items move through a lifecycle from WANTED to PURCHASED to ARCHIVED, and the dashboard aggregates total costs for budget awareness.

Supporting Features

Beyond the three pillars, Daily Organizer includes:

- **Dashboard** -- A centralized overview showing today's stats, recent activity across all three pillars, and goal progress summaries.
- **Calendar** -- A monthly grid view that plots all scheduled plans on their due dates, with a day-detail side panel for viewing and adding plans.
- **Categories** -- User-defined, color-coded labels that can be applied to both tasks and wishlist items for cross-cutting organization (e.g., "Work," "Personal," "Travel").
- **Dark/Light Theme** -- A complete dual-theme system powered by CSS custom properties, with system preference detection and localStorage persistence.
- **User Authentication** -- Secure JWT-based auth with access and refresh tokens, bcrypt password hashing, and automatic token refresh on expiry.

---

2. Technology Stack

Frontend

Technology	Version	Purpose
Angular	^19.2.0	Core framework for the single-page application
Angular Router	^19.2.0	Client-side routing with lazy loading
Angular Forms	^19.2.0	Template-driven and reactive form handling
RxJS	~7.8.0	Reactive programming for HTTP calls and async flows
TypeScript	~5.7.2	Type-safe JavaScript superset
Zone.js	~0.15.0	Angular change detection
tslib	^2.3.0	TypeScript runtime helpers
Tailwind CSS	^4.2.2	Installed but not actively used (see design decisions)
@tailwindcss/postcss	^4.2.2	PostCSS integration for Tailwind
PostCSS	^8.5.8	CSS post-processing
Autoprefixer	^10.4.27	Vendor prefix automation

Key Angular Features Used:

- **Standalone components** -- Every component is standalone (no NgModule). This is the Angular 19 best practice, reducing boilerplate and improving tree-shaking.
- **Signals** -- Angular's new reactive primitive is used for local component state and service state (e.g., `currentUser`, `theme`, `toasts`). Signals provide fine-grained reactivity without the overhead of RxJS for simple state.

- **Functional guards and interceptors** -- Route guards (`authGuard` , `noAuthGuard`) and the HTTP interceptor (`authInterceptor`) are plain functions, following the Angular 19 pattern that deprecates class-based guards.
- **CSS custom properties** -- The entire design system is built on CSS variables rather than a utility framework, enabling seamless theme switching.

Backend

Technology	Version	Purpose
NestJS Core	^11.0.1	Server framework (controllers, services, modules, DI)
NestJS Config	^4.0.3	Environment variable management (.env loading)
NestJS JWT	^11.0.2	JWT token signing and verification
NestJS Passport	^11.0.5	Authentication strategy integration
NestJS Mapped Types	^2.1.1	DTO transformation utilities (PartialType, etc.)
NestJS Platform Express	^11.0.1	Express HTTP adapter
Firebase Admin SDK	^13.7.0	Server-side Firebase Realtime Database access
Passport	^0.7.0	Authentication middleware
Passport JWT	^4.0.1	JWT extraction and validation strategy
bcrypt	^6.0.0	Password hashing (12 salt rounds)
class-validator	^0.15.1	DTO validation decorators (@IsString, @IsEmail, etc.)
class-transformer	^0.5.1	DTO transformation (used with ValidationPipe)
RxJS	^7.8.1	Reactive extensions (used internally by NestJS)
googleapis	^148.0.0	Google APIs client library (Calendar API, OAuth2)
TypeScript	^5.7.3	Type-safe JavaScript superset

Database

Technology	Purpose
Firebase Realtime Database	NoSQL JSON document store for all application data
firebase-admin SDK	Server-to-server authentication via service account

Firebase Realtime Database was chosen because it requires zero SQL schema setup, provides a generous free tier for development, stores data as a JSON tree (which maps naturally to JavaScript objects), and offers real-time capabilities that could be leveraged in the future. The `firebase-admin` SDK authenticates via a service account JSON key file, which means the backend has full read/write access without needing client-side Firebase authentication.

Authentication

Component	Purpose
JWT (Access Token)	Short-lived token (15 minutes) for API authorization
JWT (Refresh Token)	Long-lived token (7 days) for obtaining new access tokens
bcrypt	Password hashing with 12 salt rounds

Passport.js	Strategy-based authentication middleware

Styling

The application uses **pure CSS with custom properties** (CSS variables) for its entire design system. Although Tailwind CSS v4.2.2 is listed as a dependency, it is not actively used for component styling. The decision to use custom CSS instead of Tailwind is explained in the Design Decisions section.

Package Manager & Runtime

Tool	Requirement
Node.js	v18+ (LTS recommended)
npm	v9+ (ships with Node.js)
Angular CLI	^19.2.11
NestJS CLI	^11.0.0

---

3. Architecture Overview

Monorepo Structure

The project follows a monorepo layout with two top-level directories:

```
New folder/
  backend/     <-- NestJS API server (port 3000)
  frontend/    <-- Angular SPA client (port 4200)
```

Both applications are independent npm projects with their own `package.json`, `node_modules`, and build configurations. They communicate exclusively via HTTP REST API calls.

Client-Server Architecture

```
+---------------------------------------------------+
|                  BROWSER                          |
|                                                   |
|  +---------------------------------------------+ |
|  |          Angular 19 SPA (port 4200)         | |
|  |                                             | |
|  |  +---------+  +----------+  +-------------+  | |
|  |  | Auth    |  | Feature  |  | Shared      |  | |
|  |  | Pages   |  | Pages    |  | Components  |  | |
|  |  +---------+  +----------+  +-------------+  | |
|  |       |             |              |          | |
|  |  +----v-------------v--------------v------+   | |
|  |  |        Core Services Layer             |   | |
|  |  | AuthService | ThemeService | Toast ... |   | |
|  |  +----------------------------------+---+   | |
|  |                                     |       | |
|  |  +----------------------------------v---+   | |
|  |  |       HTTP Interceptor                |   | |
|  |  | Attaches Bearer token to every request |   | |
|  |  | Auto-refreshes on 401                 |   | |
|  |  +----------------------------------+---+   | |
```

```
|  +-----------------------------------------+---+ |
|                                           |     |
+-------------------------------------------+-----+
                                            |
                                       HTTP | REST
                                       JSON | /api/*
                                            |
+-------------------------------------------v-----+
|               NestJS API (port 3000)              |
|                                                   |
|  +-----------------------------------------------+  |
|  |             Global Middleware                 |  |
|  |  CORS | ValidationPipe | JwtAuthGuard (global)|  |
|  +---+-------------------------------------------+  |
|      |                                              |
|  +---v-------------------------------------------+  |
|  |             Feature Modules                   |  |
|  | AuthModule | TasksModule | GoalsModule        |  |
|  | WishlistModule | CategoriesModule             |  |
|  | DashboardModule | UsersModule                 |  |
|  +---+-------------------------------------------+  |
|      |                                              |
|  +---v-------------------------------------------+  |
|  |         PrismaModule (Global)                 |  |
|  |         FirebaseService                       |  |
|  |  push() | get() | getList() | update() | rm()|  |
|  +---+-------------------------------------------+  |
|      |                                              |
+------+----------------------------------------------+
       |
       | firebase-admin SDK
       | Service Account Auth
       |
+------v----------------------------------------------+
|         Firebase Realtime Database                   |
|                                                      |
|  users/                                              |
|    {userId}/  --> { email, username, passwordHash..} |
|  tasks/                                              |
|    {taskId}/  --> { title, type, status, dueDate.. } |
|  goals/                                              |
|    {goalId}/  --> { title, status, progress.. }      |
|  milestones/                                         |
|    {msId}/    --> { title, status, goalId, order.. } |
|  minigoals/                                          |
|    {mgId}/    --> { title, status, milestoneId.. }   |
|  wishlist/                                           |
|    {itemId}/  --> { name, price, status, currency..} |
|  categories/                                         |
|    {catId}/   --> { name, color, icon, userId.. }    |
+------------------------------------------------------+
```

Data Flow

For a typical authenticated API request, data flows through the following layers:

1. **Angular Component** calls a service method (e.g., `taskService.getAll()`)

2. **Feature Service** makes an HTTP request via `HttpClient`
3. **Auth Interceptor** clones the request and adds `Authorization: Bearer <token>`
4. **NestJS receives** the request at `http://localhost:3000/api/tasks`
5. **JwtAuthGuard** validates the Bearer token using the JWT strategy
6. **JwtStrategy** decodes the token and attaches the user object to `request.user`
7. **Controller** extracts the userId via `@CurrentUser('id')` and delegates to the service
8. **Service** calls `FirebaseService` methods to read/write data
9. **FirebaseService** communicates with Firebase Realtime Database via the Admin SDK
10. **Response** flows back through the same chain, reaching the Angular component as an Observable

JWT Authentication Flow

```
REGISTER / LOGIN
================
Client                      Server                      Firebase
  |                           |                           |
  |-- POST /api/auth/login ->|                           |
  |   { email, password }    |                           |
  |                           |-- getList('users') ------->|
  |                           |<-- [user records] ---------|
  |                           |                           |
  |                           |  bcrypt.compare(pwd, hash) |
  |                           |  generateTokens(id, email) |
  |                           |                           |
  |                           |-- update(refreshToken) --->|
  |                           |<-- ok -------------------- |
  |                           |                           |
  |<- 200 { user, tokens } --|                           |
  |                           |                           |
  |  tokens.saveTokens()     |                           |
  |  currentUser.set(user)   |                           |


AUTHENTICATED REQUEST
=====================
Client                      Server                      Firebase
  |                           |                           |
  |-- GET /api/tasks ------->|                           |
  |   Authorization: Bearer  |                           |
  |   <accessToken>          |                           |
  |                           |                           |
  |                     JwtAuthGuard                      |
  |                     JwtStrategy.validate()            |
  |                     -> extracts { sub, email }        |
  |                     -> attaches user to request       |
  |                           |                           |
  |                           |-- getList('tasks') ------->|
  |                           |<-- [task records] ---------|
  |                           |   filter by userId        |
  |                           |                           |
  |<- 200 [tasks] -----------|                           |


TOKEN REFRESH
=============
Client                      Server                      Firebase
```

```
  |                          |                            |
  |-- GET /api/tasks ------->|                            |
  |   Authorization: Bearer  |                            |
  |   <expired accessToken>  |                            |
  |                          |                            |
  |<- 401 Unauthorized ------|                            |
  |                          |                            |
  |  (interceptor catches)   |                            |
  |                          |                            |
  |-- POST /api/auth/refresh>|                            |
  |   { refreshToken }       |                            |
  |                    JwtRefreshGuard                    |
  |                    JwtRefreshStrategy.validate()      |
  |                          |                            |
  |                          |-- get('users/{id}') ----->|
  |                          |<-- user -----------------|
  |                          |  generateTokens()          |
  |                          |-- update(refreshToken) -->|
  |                          |                            |
  |<- 200 { accessToken,  ---|                            |
  |         refreshToken }   |                            |
  |                          |                            |
  |  tokens.saveTokens()     |                            |
  |                          |                            |
  |-- RETRY original req --->|  (with new accessToken)    |
  |<- 200 [tasks] -----------|                            |


LOGOUT
======
Client                     Server                      Firebase
  |                          |                            |
  |-- POST /api/auth/logout->|                            |
  |                          |-- update(refreshToken:  -->|
  |                          |    null)                   |
  |<- 200 -----------------|                            |
  |                          |                            |
  |  tokens.clearTokens()    |                            |
  |  currentUser.set(null)   |                            |
  |  router.navigate('/auth')|                            |
```

---

4. Project Structure

Backend Directory Tree

```
backend/
  .env                            # Environment variables (JWT secrets, port, DB URL)
  package.json                    # Dependencies and scripts
  tsconfig.json                   # TypeScript configuration
  nest-cli.json                   # NestJS CLI configuration
  serviceAccountKey.json          # Firebase service account credentials (gitignored)
  src/
    main.ts                       # Application entry point (bootstrap, CORS, validation)
    app.module.ts                 # Root module importing all feature modules
    common/
```

```
  decorators/
    public.decorator.ts         # @Public() decorator to bypass JWT guard
    current-user.decorator.ts   # @CurrentUser() param decorator for extracting user
prisma/
  prisma.module.ts              # Global module exporting FirebaseService
  firebase.service.ts           # Firebase Admin SDK wrapper (push, get, getList, update, remove)
auth/
  auth.module.ts                # Auth module configuration
  auth.controller.ts            # Endpoints: register, login, logout, refresh, me
  auth.service.ts               # Auth business logic: credentials, hashing, tokens
  dto/
    register.dto.ts             # Validation: email, username, password, displayName
    login.dto.ts                # Validation: email, password
  guards/
    jwt-auth.guard.ts           # Global JWT guard (checks @Public metadata)
    jwt-refresh.guard.ts        # Guard for refresh endpoint (validates refresh token)
  strategies/
    jwt.strategy.ts             # Passport strategy: extracts user from access token
    jwt-refresh.strategy.ts     # Passport strategy: extracts user from refresh token
users/
  users.module.ts               # Users module configuration
  users.controller.ts           # Endpoint: PATCH /users/me
  users.service.ts              # Profile update logic
  dto/
    update-user.dto.ts          # Validation: displayName, avatarUrl
tasks/
  tasks.module.ts               # Tasks module configuration
  tasks.controller.ts           # Endpoints: CRUD + today + upcoming + timer
  tasks.service.ts              # Task business logic: filtering, timer, categories
  dto/
    create-task.dto.ts          # Validation: title, type, priority, dueDate, etc.
    update-task.dto.ts          # Partial validation for updates
goals/
  goals.module.ts               # Goals module configuration
  goals.controller.ts           # Endpoints: CRUD goals + milestones + mini-goals
  goals.service.ts              # Goal logic: hierarchy, progress calculation
  dto/
    create-goal.dto.ts          # Validation: title, description, targetDate
    update-goal.dto.ts          # Partial validation for goal updates
    create-milestone.dto.ts     # Validation: title, description, dueDate, order
wishlist/
  wishlist.module.ts            # Wishlist module configuration
  wishlist.controller.ts        # Endpoints: CRUD + purchase
  wishlist.service.ts           # Wishlist logic: lifecycle, categories
  dto/
    create-wishlist-item.dto.ts # Validation: name, price, currency, url, imageUrl
    update-wishlist-item.dto.ts # Partial validation for updates
categories/
  categories.module.ts          # Categories module configuration
  categories.controller.ts      # Endpoints: CRUD
  categories.service.ts         # Category logic: ownership, alphabetical sort
  dto/
    create-category.dto.ts      # Validation: name, color, icon
    update-category.dto.ts      # Partial validation for updates
dashboard/
  dashboard.module.ts           # Dashboard module configuration
```

```
      dashboard.controller.ts       # Endpoints: stats, activity, calendar
      dashboard.service.ts          # Aggregation logic across tasks, goals, wishlist
```

Frontend Directory Tree

```
frontend/
  package.json                      # Dependencies and scripts
  angular.json                      # Angular CLI workspace configuration
  tsconfig.json                     # TypeScript configuration
  tsconfig.app.json                 # App-specific TS config
  tsconfig.spec.json                # Test-specific TS config
  src/
    index.html                      # Root HTML (Angular bootstraps here)
    main.ts                         # Angular entry point (bootstrapApplication)
    styles.css                      # Global design system (CSS custom properties)
    environments/
      environment.ts                # API base URL configuration
    app/
      app.component.ts              # Root component (loads currentUser on init)
      app.component.html            # Root template (<router-outlet>)
      app.component.css             # Root styles
      app.config.ts                 # Application configuration (providers, interceptor)
      app.routes.ts                 # Central route definitions with lazy loading
      core/
        guards/
          auth.guard.ts             # Protects authenticated routes (redirects to login)
          no-auth.guard.ts          # Protects auth routes (redirects to dashboard)
        interceptors/
          auth.interceptor.ts       # Attaches Bearer token, handles 401 refresh
        models/
          user.model.ts             # User interface
          task.model.ts             # Task + PlanType + PLAN_TYPES constant
          goal.model.ts             # Goal + GoalMilestone + MiniGoal interfaces
          wishlist-item.model.ts    # WishlistItem interface
          category.model.ts         # Category interface
        services/
          auth.service.ts           # Login, register, logout, refresh, currentUser signal
          token-storage.service.ts  # localStorage abstraction for JWT tokens
          theme.service.ts          # Dark/light toggle with CSS custom properties
          toast.service.ts          # Notification system with auto-dismiss
      features/
        auth/
          login/
            login.component.ts      # Login form with email/password
          register/
            register.component.ts   # Registration form with validation
        dashboard/
          dashboard.component.ts    # Today's overview: stats, timeline, goals
        tasks/
          task.service.ts           # HTTP service for task API calls
          task-list/
            task-list.component.ts  # Plan list with type/status/priority filters
          task-detail/
            task-detail.component.ts# Single plan view with timer controls
          task-form/
            task-form.component.ts  # Create/edit plan form with type selector
```

```
      goals/
        goal.service.ts           # HTTP service for goal API calls
        goal-list/
          goal-list.component.ts  # Goal cards with progress bars
        goal-detail/
          goal-detail.component.ts# Milestones + mini-goals nested view
        goal-form/
          goal-form.component.ts  # Create/edit goal form
      wishlist/
        wishlist.service.ts       # HTTP service for wishlist API calls
        wishlist-list/
          wishlist-list.component.ts # Product cards with images and prices
        wishlist-form/
          wishlist-form.component.ts # Create/edit item form with currency
      calendar/
        calendar.component.ts     # Monthly grid with day-detail panel
      categories/
        category.service.ts       # HTTP service for category API calls
    shared/
      components/
        layout/
          layout.component.ts     # App shell: sidebar + content area
        sidebar/
          sidebar.component.ts    # Navigation, categories, theme toggle, user info
        topbar/
          topbar.component.ts     # Page title bar with theme toggle and avatar
        modal/
          modal.component.ts      # Generic dialog wrapper (backdrop + centered box)
        toast-container/
          toast-container.component.ts # Renders active toast notifications
        confirm-dialog/
          confirm-dialog.component.ts # Styled delete confirmation popup (replaces browser confirm())
        category-manager/
          category-manager.component.ts # Popup form for creating/editing categories
```

---

5. Backend Deep Dive

5.1 NestJS Module System

NestJS organizes applications into modules, and Daily Organizer follows this pattern with a root `AppModule` that imports all feature modules.

AppModule (`src/app.module.ts`) serves as the root of the application and performs three key responsibilities:

1. **Loads environment variables globally** via `ConfigModule.forRoot({ isGlobal: true })` , which reads the `.env` file and makes all variables available through `ConfigService` in any module.
2. **Imports all feature modules**: `AuthModule` , `UsersModule` , `TasksModule` , `GoalsModule` , `WishlistModule` , `CategoriesModule` , `DashboardModule` , and `PrismaModule` .
3. **Registers a global guard** via the `APP_GUARD` token, which makes `JwtAuthGuard` active on every route in the application. Routes that need to be publicly accessible must use the `@Public()` decorator to opt out.

```
@Module({
  imports: [
    ConfigModule.forRoot({ isGlobal: true }),
```

```
    PrismaModule,       // Global -- FirebaseService available everywhere
    AuthModule,          // /api/auth/*
    UsersModule,         // /api/users/*
    TasksModule,         // /api/tasks/*
    GoalsModule,         // /api/goals/*
    WishlistModule,      // /api/wishlist/*
    CategoriesModule,    // /api/categories/*
    DashboardModule,     // /api/dashboard/*
  ],
  providers: [
    { provide: APP_GUARD, useClass: JwtAuthGuard },
  ],
})
export class AppModule {}
```

PrismaModule (`src/prisma/prisma.module.ts`) is a global module that exports `FirebaseService` . By marking it as global, any service in any module can inject `FirebaseService` without explicitly importing `PrismaModule` . The name "Prisma" is a vestigial reference from an earlier design that used Prisma ORM; the module was repurposed for Firebase without renaming.

Feature modules each follow the standard NestJS pattern:

- A module file that declares its controller and service
- A controller that defines route handlers and delegates to the service
- A service that contains business logic and calls FirebaseService
- DTOs (Data Transfer Objects) that define and validate request body shapes

5.2 Firebase Integration (FirebaseService)

The `FirebaseService` (`src/prisma/firebase.service.ts`) is the sole database access layer for the entire application. It wraps the Firebase Admin SDK and provides a set of generic CRUD helper methods that all feature services use.

Why Firebase Realtime Database?

Firebase Realtime Database was chosen for several pragmatic reasons:

- **Zero schema setup** -- Unlike SQL databases, there is no need to define tables, columns, migrations, or relationships. Data is stored as a JSON tree, which maps directly to JavaScript/TypeScript objects.
- **Free tier** -- Firebase's Spark plan provides 1 GB of storage and 10 GB of monthly data transfer at no cost, which is more than sufficient for a personal organizer application.
- **Simple CRUD** -- The Admin SDK provides straightforward `set()` , `get()` , `update()` , and `remove()` methods that map to standard REST operations.
- **Service account authentication** -- The backend authenticates with Firebase using a service account JSON key, giving full read/write access without client-side authentication.

Initialization (OnModuleInit)

When the NestJS application starts, `FirebaseService.onModuleInit()` runs automatically:

1. Resolves the path to `serviceAccountKey.json` (located at the backend root)
2. Loads the service account credentials
3. Initializes the Firebase Admin app with the credentials and the database URL
4. Stores the database reference for later use

```
onModuleInit() {
  const serviceAccountPath = path.resolve(__dirname, '../../serviceAccountKey.json');
  const serviceAccount = require(serviceAccountPath);
  this.app = admin.initializeApp({
    credential: admin.credential.cert(serviceAccount),
```

```
    databaseURL: 'https://taskmanagement-ce2c2-default-rtdb.firebaseio.com',
  });
  this._db = this.app.database();
}
```

CRUD Helper Methods

Method	Signature	Description
`ref()`	`ref(path: string): Reference`	Returns a raw Firebase database reference for advanced queries
`push()`	`push<T>(path, data): Promise<T & { id }>`	Creates a record with auto-generated push key, adds `id`, `createdAt`, `updatedAt`
`get()`	`get<T>(path): Promise<T | null>`	Reads a single record by path; returns `null` if not found
`getList()`	`getList<T>(path): Promise<T[]>`	Reads all children under a path as an array
`update()`	`update(path, data): Promise<void>`	Merges fields into an existing record; auto-sets `updatedAt`
`remove()`	`remove(path): Promise<void>`	Deletes a record entirely

Firebase Data Structure

Data is organized under top-level collection paths in the Firebase Realtime Database:

```
root/
  users/
    {userId}/         # User account data (email, username, passwordHash, refreshToken...)
  tasks/
    {taskId}/         # Task/plan records (title, type, status, dueDate, timer fields...)
  goals/
    {goalId}/         # Goal records (title, status, progress, targetDate...)
  milestones/
    {milestoneId}/    # Milestone records (title, status, goalId, order...)
  minigoals/
    {miniGoalId}/     # Mini-goal records (title, status, milestoneId, goalId...)
  wishlist/
    {itemId}/         # Wishlist item records (name, price, currency, status...)
  categories/
    {categoryId}/     # Category records (name, color, icon, userId...)
```

Note that milestones and mini-goals are stored in their own top-level collections (not nested under goals) and use foreign keys (`goalId` , `milestoneId`) for relationships. This flat structure makes it easy to query and delete individual records without navigating nested paths.

5.3 Authentication System

The authentication system uses JWT (JSON Web Tokens) with a dual-token strategy: a short-lived access token for API authorization and a long-lived refresh token for session renewal.

Registration Flow (Step by Step)

1. Client sends `POST /api/auth/register` with `{ email, username, password, displayName? }`
2. `RegisterDto` validates the request body (all fields must be strings, email must be valid)
3. `AuthService.register()` fetches all users from Firebase via `getList('users')`
4. Checks for duplicate email or username; throws `BadRequestException` if found
5. Generates a new UUID via `crypto.randomUUID()` for the user ID
6. Hashes the plaintext password with `bcrypt.hash(password, 12)` (12 salt rounds)
7. Creates the user record object with `id` , `email` , `username` , `passwordHash` , `displayName` , `avatarUrl: null` , `refreshToken: null` , timestamps

8. Writes the user to Firebase at `users/{id}`
9. Generates an access token and refresh token via `generateTokens(id, email)`
10. Stores the refresh token in the user's Firebase record
11. Returns `{ user: { id, email, username, displayName, avatarUrl }, accessToken, refreshToken }`

Login Flow (Step by Step)

1. Client sends `POST /api/auth/login` with `{ email, password }`
2. `LoginDto` validates the request body
3. `AuthService.login()` fetches all users and finds the one matching the email
4. If no user found, throws `UnauthorizedException('Invalid credentials')`
5. Compares the password against the stored hash with `bcrypt.compare(password, passwordHash)`
6. If mismatch, throws `UnauthorizedException('Invalid credentials')`
7. Generates new access and refresh tokens
8. Updates the user's refresh token in Firebase
9. Returns user profile + both tokens

Token Generation

Both tokens are signed concurrently using `Promise.all` for performance:

```
private async generateTokens(userId: string, email: string) {
  const payload = { sub: userId, email };
  const [accessToken, refreshToken] = await Promise.all([
    this.jwt.signAsync(payload, {
      secret: this.config.get('JWT_SECRET'),
      expiresIn: this.config.get('JWT_EXPIRES_IN'),     // 15m
    }),
    this.jwt.signAsync(payload, {
      secret: this.config.get('JWT_REFRESH_SECRET'),
      expiresIn: this.config.get('JWT_REFRESH_EXPIRES_IN'), // 7d
    }),
  ]);
  return { accessToken, refreshToken };
}
```

The access token uses `JWT_SECRET` with a 15-minute expiry. The refresh token uses a separate `JWT_REFRESH_SECRET` with a 7-day expiry. Using different secrets ensures that an access token cannot be used as a refresh token and vice versa.

Token Refresh Flow

1. Client's access token expires, causing a 401 response
2. Frontend interceptor catches the 401 and sends `POST /api/auth/refresh` with `{ refreshToken }`
3. The endpoint uses `@Public()` to bypass the global JWT guard (since the access token is expired)
4. `JwtRefreshGuard` validates the refresh token using `JwtRefreshStrategy`
5. `AuthService.refreshTokens()` fetches the user, generates new tokens, rotates the stored refresh token
6. Returns new `{ accessToken, refreshToken }` pair

Logout Flow

1. Client sends `POST /api/auth/logout` (requires valid access token)
2. `AuthService.logout()` sets `refreshToken: null` in the user's Firebase record
3. This invalidates the refresh token, preventing future token renewal
4. The frontend clears tokens from localStorage and redirects to login

JwtStrategy vs JwtRefreshStrategy

- **JwtStrategy** -- Extracts the Bearer token from the `Authorization` header, validates it against `JWT_SECRET`, and attaches the decoded user to the request. Used by the global `JwtAuthGuard` on all protected routes.

- **JwtRefreshStrategy** -- Extracts the refresh token from the request body, validates it against `JWT_REFRESH_SECRET`. Used only by the `JwtRefreshGuard` on the `/auth/refresh` endpoint.

@Public() Decorator Mechanism

The `@Public()` decorator uses `SetMetadata` to attach `{ isPublic: true }` to a route handler. The global `JwtAuthGuard` checks for this metadata via the Reflector. If present, the guard allows the request through without JWT validation. This pattern means all routes are protected by default, and only explicitly marked routes are publicly accessible. Currently, three routes are public: `POST /auth/register`, `POST /auth/login`, and `POST /auth/refresh`.

@CurrentUser() Decorator Mechanism

The `@CurrentUser()` decorator is a custom parameter decorator that extracts the user object from `request.user` (set by Passport after JWT validation). It supports an optional property name argument: `@CurrentUser('id')` returns just the user's ID string, while `@CurrentUser()` returns the full user object.

5.4 Tasks/Plans Module

The tasks module is the most feature-rich module in the application, implementing the multi-type plan system that differentiates Daily Organizer from a standard to-do app.

Multi-Type Plan System

Every plan in the system has a `type` field that classifies it into one of seven categories:

Type	Icon	Color	Typical Use
`task`	Check mark	Blue #3b82f6	General to-do items, work tasks
`trip`	Airplane	Green #10b981	Travel plans, vacations
`train`	Train	Amber #f59e0b	Commute schedules, rail travel
`dinner`	Plate	Pink #ec4899	Dinner reservations, meal plans
`meeting`	People	Purple #8b5cf6	Meetings, calls, interviews
`event`	Calendar	Cyan #06b6d4	Generic events, parties, conferences
`reminder`	Bell	Orange #f97316	Reminders, alerts, deadlines

All seven types share the same data model and API endpoints. The `type` field determines how the plan is displayed in the UI (icon, color, filter chip) and which context-specific form fields are shown. While the underlying data model is shared, each type uses different subsets of fields -- for example, train plans use `boardingStation`, `destinationStation`, `trainNumber`, and `departureTime`, while meeting plans use `meetingLink` and `startTime` / `endTime`.

CRUD Operations

- **Create** (`POST /api/tasks`) -- Takes a `CreateTaskDto` with required `title` and optional fields (type defaults to `task`, status to `TODO`, priority to `MEDIUM`). Generates a UUID, builds the full record with defaults, writes to Firebase, and returns the task with its category attached.
- **Read All** (`GET /api/tasks`) -- Fetches all tasks from Firebase, filters by `userId`, then applies optional query filters for `status`, `priority`, `categoryId`, and `type`. Each result is enriched with its category object.
- **Read One** (`GET /api/tasks/:id`) -- Fetches a single task, verifies ownership, enriches with category.
- **Update** (`PATCH /api/tasks/:id`) -- Verifies ownership, applies partial updates. Handles status transitions: if status changes to `DONE`, sets `completedAt` to current timestamp; if status changes to anything else, clears `completedAt`.
- **Delete** (`DELETE /api/tasks/:id`) -- Verifies ownership, removes from Firebase.

Filtering

The `findAll` method supports four query parameters that can be combined:

```
GET /api/tasks?status=TODO&priority=HIGH&type=meeting&categoryId=abc123
```

Filtering is performed in-memory after fetching all tasks from Firebase. This approach works well for personal organizer data volumes (hundreds to low thousands of tasks per user) but would need indexing for larger datasets.

Today and Upcoming Queries

- `GET /api/tasks/today` -- Returns tasks whose `dueDate` starts with today's date string (YYYY-MM-DD prefix matching).
- `GET /api/tasks/upcoming` -- Returns tasks due within the next 7 days that are not yet marked as `DONE`.

Timer System

Each task has a built-in timer for time tracking:

- **Start** (`POST /api/tasks/:id/timer/start`) -- Sets `isTimerActive: true` and `timerStartAt` to the current ISO timestamp.
- **Stop** (`POST /api/tasks/:id/timer/stop`) -- Calculates elapsed minutes since `timerStartAt`, adds them to `trackedMins`, sets `isTimerActive: false`, and clears `timerStartAt`.

The elapsed time calculation uses:

```
const elapsed = Math.floor((Date.now() - new Date(task.timerStartAt).getTime()) / 60000);
```

This gives whole minutes elapsed. The cumulative `trackedMins` field persists across multiple start/stop cycles, providing a total time tracked for the task.

Category Enrichment

Tasks can optionally belong to a category. The service provides two private helpers:

- `withCategory(task)` -- If the task has a `categoryId`, fetches the category from Firebase and attaches it as `task.category`. Otherwise sets `category: null`.
- `withCategories(tasks)` -- Maps over an array of tasks and calls `withCategory` on each, using `Promise.all` for concurrent execution.

Ownership Verification

The `ensureOwnership(userId, id)` pattern is used by update, delete, and timer operations:

1. Fetch the task from Firebase
2. If not found, throw `NotFoundException`
3. If `task.userId !== userId`, throw `ForbiddenException`
4. Return the task data (avoids redundant reads in the caller)

5.5 Goals Module

The goals module implements a three-level hierarchy for long-term goal tracking.

Three-Level Hierarchy

```
Goal (e.g., "Learn SQL")
  |
  +-- Milestone (e.g., "Complete Basic Queries Course")
  |     |
  |     +-- Mini-Goal (e.g., "Practice SELECT statements")
  |     +-- Mini-Goal (e.g., "Learn JOIN syntax")
  |
```

```
+-- Milestone (e.g., "Build a Practice Database")
      |
      +-- Mini-Goal (e.g., "Design schema")
      +-- Mini-Goal (e.g., "Insert sample data")
```

- **Goal** -- The top-level objective. Has a status (ACTIVE, COMPLETED, PAUSED, ABANDONED), an optional target date, and an auto-calculated progress percentage.
- **Milestone** -- A major checkpoint within a goal. Has a display order, optional due date, and a status (PENDING or COMPLETED).
- **Mini-Goal** -- A small actionable checklist item within a milestone. Has a toggle-able status (PENDING/COMPLETED).

Progress Auto-Calculation

When a milestone is marked as completed via `PATCH /api/goals/:goalId/milestones/:milestoneId/complete`, the service recalculates the goal's progress:

```
const milestones = await this.getMilestones(goalId);
const completed = milestones.filter((m) => m.status === 'COMPLETED').length;
const progress = milestones.length > 0 ? (completed / milestones.length) * 100 : 0;
await this.firebase.update(`goals/${goalId}`, { progress });
```

Progress is stored as a number from 0 to 100, representing the percentage of milestones completed. Note that mini-goal completion does not directly affect the goal's progress -- only milestone completion does.

CRUD for Each Level

Goals:

- `GET /api/goals` -- All goals for the user, each with milestones and mini-goals attached
- `GET /api/goals/:id` -- Single goal with full hierarchy
- `POST /api/goals` -- Create a new goal (progress starts at 0)
- `PATCH /api/goals/:id` -- Update goal fields
- `DELETE /api/goals/:id` -- Delete goal and cascade-delete all milestones

Milestones:

- `POST /api/goals/:id/milestones` -- Add a milestone to a goal
- `PATCH /api/goals/:goalId/milestones/:milestoneId` -- Update milestone fields
- `PATCH /api/goals/:goalId/milestones/:milestoneId/complete` -- Mark as completed, recalculate progress
- `DELETE /api/goals/:goalId/milestones/:milestoneId` -- Delete milestone and its mini-goals

Mini-Goals:

- `POST /api/goals/:goalId/milestones/:milestoneId/minigoals` -- Add a mini-goal
- `PATCH /api/goals/:goalId/minigoals/:miniGoalId/toggle` -- Toggle PENDING/COMPLETED
- `DELETE /api/goals/:goalId/minigoals/:miniGoalId` -- Remove a mini-goal

Cascade Deletion

When a goal is deleted:

1. All milestones belonging to the goal are fetched
2. Each milestone is deleted from `milestones/` collection
3. The goal itself is deleted from `goals/` collection

When a milestone is deleted:

1. All mini-goals belonging to the milestone are fetched
2. Each mini-goal is deleted from `minigoals/` collection
3. The milestone itself is deleted from `milestones/` collection

Milestone Attachment

The `attachMilestones(goal)` helper:

1. Fetches all milestones from Firebase, filters by `goalId`, sorts by `order`
2. For each milestone, fetches all mini-goals filtered by `milestoneId`
3. Attaches the nested structure to the goal object

5.6 Wishlist (Buy List) Module

The wishlist module provides a shopping list / buy list feature where users track products they want to purchase.

Item Lifecycle

```
WANTED  -->  PURCHASED  -->  ARCHIVED
(initial)   (bought)       (hidden from active list)
```

- **WANTED** -- The default status when an item is created. The item appears in the active buy list.
- **PURCHASED** -- Set via the dedicated `PATCH /api/wishlist/:id/purchase` endpoint, which also records `purchasedAt` timestamp.
- **ARCHIVED** -- Can be set via the general update endpoint for items the user no longer wants to track.

Price Tracking with Currency

Each item has optional `price` and `currency` fields. The `currency` defaults to `'USD'` if not specified. The dashboard service aggregates the total USD cost of all WANTED items for the user's overview.

Supported currencies in the form: USD, EUR, GBP, INR (the currency field accepts any string, but the form provides these four options).

Purchase Endpoint

The `purchase` method is a dedicated endpoint (`PATCH /api/wishlist/:id/purchase`) that:

1. Verifies ownership
2. Updates status to `'PURCHASED'`
3. Sets `purchasedAt` to the current ISO timestamp
4. Returns the updated item

This is separate from the general update endpoint to provide a clean, semantic action for the "Mark as Purchased" button in the UI.

5.7 Categories Module

Categories provide color-coded organizational labels that can be applied to both tasks and wishlist items.

Color-Coded Organization

Each category has:

- `name` -- Display label (e.g., "Work," "Personal," "Travel")
- `color` -- Hex color string (defaults to `#6366f1` indigo if not specified)
- `icon` -- Optional emoji or icon string (reserved for future UI use)

Categories are displayed as colored dots in the sidebar navigation and as colored tags on task/wishlist items throughout the app.

Used by Tasks AND Wishlist Items

Both the task and wishlist item models have a `categoryId` foreign key. When fetching tasks or wishlist items, the respective services look up the category by ID and attach the full category object to each record. This allows the frontend to display the category name and color without a separate API call.

Ownership Pattern

Categories are per-user. Each category has a `userId` field, and all operations verify that the requesting user owns the category before allowing reads, updates, or deletes. The `findAll` method filters by `userId` and sorts alphabetically by name.

5.8 Dashboard Module

The dashboard module aggregates data from tasks, goals, and wishlist to provide an overview for the user's landing page.

Stats Aggregation

`GET /api/dashboard/stats` returns:

Stat	Calculation
`totalTasks`	Count of all tasks belonging to the user
`completedToday`	Tasks with `status === 'DONE'` AND `completedAt` starting with today's date
`activeTasks`	Tasks with `status !== 'DONE'` (includes TODO and IN_PROGRESS)
`activeGoals`	Goals with `status === 'ACTIVE'` (excludes COMPLETED, PAUSED, ABANDONED)
`wishlistCount`	Wishlist items with `status === 'WANTED'` (not yet purchased)
`wishlistTotalUSD`	Sum of `price` for WANTED items where `currency === 'USD'`, rounded to 2 decimals

Activity Feed

`GET /api/dashboard/activity` returns the 10 most recent updates across all three pillars:

1. Takes up to 5 recent tasks (mapped to `{ type: 'task', id, title, status, updatedAt }`)
2. Takes up to 3 recent goals (mapped to `{ type: 'goal', id, title, status, updatedAt }`)
3. Takes up to 2 recent wishlist items (mapped to `{ type: 'wishlist', id, title, status, updatedAt }`)
4. Merges all items into a single array
5. Sorts by `updatedAt` descending (newest first)
6. Returns the top 10 items

Calendar Data

`GET /api/dashboard/calendar?year=2026&month=4` returns tasks for a specific month:

1. Fetches all tasks from Firebase
2. Filters by `userId`, `dueDate` existence, and matching year/month
3. Maps each task to a calendar-friendly format with `id`, `title`, `type`, `status`, `priority`, `dueDate`, `startTime`, `endTime`, `location`
4. The `category` field is set to `null` in calendar responses to keep payloads lightweight

5.9 API Reference

All endpoints are prefixed with `/api`. Authentication is required unless noted.

Auth Endpoints

Method	Path	Auth	Description	Request Body	Response
`POST`	`/api/auth/register`	Public	Create a new account	`{ email, username, password, displayName? }`	`{ user, accessToken, refreshToken }`
`POST`	`/api/auth/login`	Public	Log in with credentials	`{ email, password }`	`{ user, accessToken, refreshToken }`
`POST`	`/api/auth/logout`	JWT	Invalidate refresh token	None	`void`
`POST`	`/api/auth/refresh`	Refresh	Get new token pair	`{ refreshToken }`	`{ accessToken, refreshToken }`

GET	/api/auth/me	JWT	Get current user profile	None	User

User Endpoints

Method	Path	Auth	Description	Request Body	Response
PATCH	/api/users/me	JWT	Update own profile	{ displayName?, avatarUrl? }	User

Task Endpoints

Method	Path	Auth	Description	Request Body / Query	Response
GET	/api/tasks	JWT	List all tasks (filtered)	Query: status, priority, categoryId, type	Task[]
GET	/api/tasks/today	JWT	Tasks due today	None	Task[]
GET	/api/tasks/upcoming	JWT	Tasks due in next 7 days	None	Task[]
GET	/api/tasks/:id	JWT	Get single task	None	Task
POST	/api/tasks	JWT	Create a new task/plan	CreateTaskDto	Task
PATCH	/api/tasks/:id	JWT	Update a task	UpdateTaskDto (partial)	Task
DELETE	/api/tasks/:id	JWT	Delete a task	None	{ deleted: true }
POST	/api/tasks/:id/timer/start	JWT	Start timer	None	Task
POST	/api/tasks/:id/timer/stop	JWT	Stop timer, add elapsed	None	Task

Goal Endpoints

Method	Path	Auth	Description	Request Body	Response
GET	/api/goals	JWT	List all goals	None	Goal[] (with milestones + mini-goals)
GET	/api/goals/:id	JWT	Get single goal	None	Goal (with milestones + mini-goals)
POST	/api/goals	JWT	Create a goal	{ title, description?, status?, targetDate? }	Goal
PATCH	/api/goals/:id	JWT	Update a goal	Partial goal fields	Goal
DELETE	/api/goals/:id	JWT	Delete goal + milestones	None	{ deleted: true }
POST	/api/goals/:id/milestones	JWT	Add milestone	{ title, description?, dueDate?, order? }	Goal

PATCH	/api/goals/:gId/milestones/:mId	JWT	Update milestone	Partial milestone fields	Milestone
PATCH	/api/goals/:gId/milestones/:mId/complete	JWT	Complete milestone	None	Goal (with updated progress)
DELETE	/api/goals/:gId/milestones/:mId	JWT	Delete milestone + mini-goals	None	{ deleted: true }
POST	/api/goals/:gId/milestones/:mId/minigoals	JWT	Add mini-goal	{ title }	Goal
PATCH	/api/goals/:gId/minigoals/:mgId/toggle	JWT	Toggle mini-goal status	None	Goal
DELETE	/api/goals/:gId/minigoals/:mgId	JWT	Delete mini-goal	None	Goal

Wishlist Endpoints

Method	Path	Auth	Description	Request Body / Query	Response
GET	/api/wishlist	JWT	List all items (filtered)	Query: status, priority, categoryId	WishlistItem[]
GET	/api/wishlist/:id	JWT	Get single item	None	WishlistItem
POST	/api/wishlist	JWT	Create item	CreateWishlistItemDto	WishlistItem
PATCH	/api/wishlist/:id	JWT	Update item	UpdateWishlistItemDto (partial)	WishlistItem
PATCH	/api/wishlist/:id/purchase	JWT	Mark as purchased	None	WishlistItem
DELETE	/api/wishlist/:id	JWT	Delete item	None	{ deleted: true }

Category Endpoints

Method	Path	Auth	Description	Request Body	Response
GET	/api/categories	JWT	List all categories (sorted)	None	Category[]
POST	/api/categories	JWT	Create category	{ name, color?, icon? }	Category
PATCH	/api/categories/:id	JWT	Update category	Partial fields	Category
DELETE	/api/categories/:id	JWT	Delete category	None	{ deleted: true }

Dashboard Endpoints

Method	Path	Auth	Description	Query	Response
GET	/api/dashboard/stats	JWT	Aggregated statistics	None	{ totalTasks, completedToday, activeTasks, activeGoals, wishlistCount, wishlistTotalUSD }
GET	/api/dashboard/activity	JWT	Recent activity feed	None	ActivityItem[] (max 10)
GET	/api/dashboard/calendar	JWT	Monthly calendar data	year, month (default: current)	CalendarTask[]

Google Calendar Endpoints

Method	Path	Auth	Description	Request Body	Response
`GET`	`/api/google/auth`	JWT	Returns Google OAuth consent URL	None	`{ url: string }`
`GET`	`/api/google/callback`	Public	OAuth callback, stores tokens, redirects to frontend	None	Redirect to `/?gcal=connected`
`GET`	`/api/google/status`	JWT	Check Google Calendar connection status	None	`{ connected: boolean }`
`POST`	`/api/google/disconnect`	JWT	Removes Google tokens from user record	None	`{ disconnected: true }`
`POST`	`/api/google/sync-all`	JWT	Syncs all unsynced tasks (with dueDate, without googleEventId) to Google Calendar	None	`{ synced: number }`

---

6. Frontend Deep Dive

6.1 Angular Architecture

The frontend is built with Angular 19 and follows modern best practices throughout.

Standalone Components (No NgModule)

Every component in the application is declared as `standalone: true`. This means there are no `NgModule` declarations anywhere in the codebase. Instead, each component imports the Angular directives and pipes it needs directly in its own `imports` array. This approach, which became the recommended pattern in Angular 17+, provides several benefits:

- Reduced boilerplate (no module files for feature areas)
- Better tree-shaking (unused components are not bundled)
- Clearer dependency graphs (each component declares its own imports)
- Simpler lazy loading (load a component directly, not a module)

Signals for Reactive State

Angular 19 signals are used extensively for local and shared state:

- `AuthService.currentUser` -- A `signal<User | null>` that holds the authenticated user's profile. Updated on login/register and cleared on logout. Components read this signal reactively in their templates.
- `ThemeService.theme` -- A `signal<Theme>` that holds `'light'` or `'dark'`. Read by the sidebar and topbar to display the correct toggle icon.
- `ToastService.toasts` -- A `signal<Toast[]>` holding the array of currently visible notifications. The toast-container component reads this to render/remove toasts.

Signals were chosen over BehaviorSubject for these use cases because they provide simpler, synchronous reads without the overhead of subscriptions and unsubscription.

BehaviorSubject for Shared Service State

While signals handle simple state, RxJS `BehaviorSubject` (via `HttpClient` Observables) is still used for HTTP data flows where operators like `tap`, `switchMap`, and `catchError` are needed. The auth interceptor, for example, uses RxJS operators extensively for its 401 retry logic.

Lazy Loading with loadComponent

Every feature page is lazy-loaded via `loadComponent` in the route configuration. This means the browser only downloads the JavaScript for a page when the user navigates to it:

```
{
  path: 'dashboard',
  loadComponent: () =>
    import('./features/dashboard/dashboard.component')
      .then((m) => m.DashboardComponent),
}
```

This keeps the initial bundle small (only the auth pages and core services are loaded eagerly) and improves time-to-interactive.

bootstrapApplication Pattern

The application is bootstrapped without an `AppModule` using:

```
bootstrapApplication(AppComponent, appConfig);
```

where `appConfig` provides the router, HTTP client, and interceptor via `provideRouter`, `provideHttpClient`, and `withInterceptors`.

6.2 Routing System

The routing configuration in `app.routes.ts` defines three groups of routes:

1. Root Redirect

```
{ path: '', redirectTo: 'dashboard', pathMatch: 'full' }
```

Navigating to the bare URL `/` redirects to `/dashboard`.

2. Auth Routes (No Layout)

```
{
  path: 'auth',
  canActivate: [noAuthGuard],
  children: [
    { path: 'login', loadComponent: () => import('...LoginComponent') },
    { path: 'register', loadComponent: () => import('...RegisterComponent') },
    { path: '', redirectTo: 'login', pathMatch: 'full' },
  ],
}
```

The `noAuthGuard` prevents already-logged-in users from seeing the login/register pages by redirecting them to `/dashboard`. These routes render without the sidebar/layout wrapper, giving a clean full-screen auth experience.

3. Authenticated Routes (With Layout)

```
{
  path: '',
  canActivate: [authGuard],
  loadComponent: () => import('...LayoutComponent'),
  children: [
    { path: 'dashboard', loadComponent: () => import('...DashboardComponent') },
    { path: 'tasks', loadComponent: () => import('...TaskListComponent') },
    { path: 'tasks/:id', loadComponent: () => import('...TaskDetailComponent') },
    { path: 'goals', loadComponent: () => import('...GoalListComponent') },
    { path: 'goals/:id', loadComponent: () => import('...GoalDetailComponent') },
```

```
    { path: 'wishlist', loadComponent: () => import('...WishlistListComponent') },
    { path: 'calendar', loadComponent: () => import('...CalendarComponent') },
  ],
}
```

The `authGuard` checks for a stored access token and redirects to `/auth/login` if none is found. The `LayoutComponent` wraps all children in the app shell (sidebar + topbar + content area).

4. Wildcard

```
{ path: '**', redirectTo: 'dashboard' }
```

Any unmatched URL redirects to the dashboard.

Route Hierarchy Diagram

```
/
  --> /dashboard (redirect)

/auth
  [noAuthGuard]
  /auth/login       --> LoginComponent
  /auth/register    --> RegisterComponent

/ (empty path, LayoutComponent wrapper)
  [authGuard]
  /dashboard        --> DashboardComponent
  /tasks            --> TaskListComponent
  /tasks/:id        --> TaskDetailComponent
  /goals            --> GoalListComponent
  /goals/:id        --> GoalDetailComponent
  /wishlist         --> WishlistListComponent
  /calendar         --> CalendarComponent

/** (wildcard)
  --> /dashboard (redirect)
```

6.3 Core Services

AuthService

`core/services/auth.service.ts`

The central authentication service, provided in root (singleton). Key members:

Member	Type	Description
`currentUser`	`signal<User | null>`	Reactive signal holding the current user profile
`register(dto)`	Method	Sends registration data, stores tokens, sets user signal
`login(dto)`	Method	Sends credentials, stores tokens, sets user signal
`logout()`	Method	Calls backend logout, clears tokens, redirects to login
`refresh()`	Method	Sends refresh token, updates stored tokens
`loadCurrentUser()`	Method	Calls `GET /auth/me`, updates user signal

`isLoggedIn()`	Method	Synchronous check for stored access token

The `handleAuthResponse` private method is shared between `login` and `register` : it saves both tokens via `TokenStorageService` and updates the `currentUser` signal. The `clearSession` private method is shared between `logout` and failed refresh: it clears tokens, resets the signal to `null` , and navigates to `/auth/login` .

TokenStorageService

`core/services/token-storage.service.ts`

A thin abstraction over `localStorage` that centralizes token storage:

Method	Description
`saveTokens(access, refresh)`	Saves both tokens to localStorage
`getAccessToken()`	Returns the access token or null
`getRefreshToken()`	Returns the refresh token or null
`clearTokens()`	Removes both tokens

Storage keys: `tf_access` for the access token, `tf_refresh` for the refresh token. Centralizing storage here makes it easy to swap mechanisms (e.g., to sessionStorage or secure cookies) without changing callers.

ThemeService

`core/services/theme.service.ts`

Manages the dark/light theme across the entire application:

Member	Type	Description
`theme`	`signal<Theme>`	Current theme ('light' or 'dark')
`toggle()`	Method	Switches between light and dark

On initialization, the service:

1. Checks localStorage for a saved preference (key: `tf_theme`)
2. If none, checks `window.matchMedia('(prefers-color-scheme: dark)')` for the OS preference
3. Applies whichever theme was found via `applyTheme()`

The `applyTheme` method:

1. Updates the reactive signal
2. Sets `data-theme` attribute on `document.documentElement` (which triggers CSS variable swaps)
3. Persists the choice to localStorage

ToastService

`core/services/toast.service.ts`

Manages ephemeral notification messages:

Member	Description
`toasts`	`signal<Toast[]>` -- array of active toast notifications
`show(message, type, duration)`	Creates a toast and schedules auto-dismissal
`success(msg)`	Shorthand for green success toast (3.5s)

`error(msg)`	Shorthand for red error toast (5s for readability)
`info(msg)`	Shorthand for blue info toast (3.5s)
`warning(msg)`	Shorthand for amber warning toast (3.5s)
`dismiss(id)`	Removes a specific toast by ID

Each toast gets a monotonically increasing ID from an internal counter. The auto-dismiss timeout calls `dismiss(id)` after the specified duration. Users can also click a toast to dismiss it immediately.

6.4 HTTP Interceptor

`core/interceptors/auth.interceptor.ts`

The auth interceptor is a functional HTTP interceptor (Angular 19 pattern) that handles two responsibilities:

1. Attaching Bearer Tokens

For every outgoing HTTP request, the interceptor:

1. Reads the access token from `TokenStorageService`
2. If a token exists, clones the request with an `Authorization: Bearer <token>` header
3. If no token exists, forwards the original request unchanged

2. Automatic Token Refresh on 401

When a 401 response is received:

1. Checks three conditions: the error is 401, no refresh is already in progress (`isRefreshing` flag), and the failing request is NOT an auth endpoint (prevents infinite loops)
2. Sets `isRefreshing = true` to block concurrent refresh attempts
3. Calls `auth.refresh()` which sends the refresh token to the backend
4. On success: resets the flag, retries the original request with the new access token
5. On failure: resets the flag, calls `auth.logout()` (session expired), re-throws the error

The `isRefreshing` module-level flag is critical for preventing multiple simultaneous refresh requests when several API calls return 401 at the same time (e.g., when navigating to a page that makes multiple parallel requests).

```
let isRefreshing = false;

export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const tokens = inject(TokenStorageService);
  const auth = inject(AuthService);
  const token = tokens.getAccessToken();
  const authReq = token
    ? req.clone({ setHeaders: { Authorization: `Bearer ${token}` } })
    : req;

  return next(authReq).pipe(
    catchError((err: HttpErrorResponse) => {
      if (err.status === 401 && !isRefreshing && !req.url.includes('/auth/')) {
        isRefreshing = true;
        return auth.refresh().pipe(
          switchMap((res) => {
            isRefreshing = false;
            const retried = req.clone({
              setHeaders: { Authorization: `Bearer ${res.accessToken}` }
            });
```

```
            return next(retried);
          }),
          catchError((refreshErr) => {
            isRefreshing = false;
            auth.logout();
            return throwError(() => refreshErr);
          }),
        );
      }
      return throwError(() => err);
    }),
  );
};
```

6.5 Design System (styles.css)

The entire visual design of the application is built on CSS custom properties defined in `src/styles.css`. This file serves as the design system, defining colors, typography, spacing, and reusable component classes.

Why CSS Custom Properties (Not Tailwind)

Although Tailwind CSS v4.2.2 is installed as a dependency, the application uses hand-written CSS with custom properties instead. Tailwind v4 introduced a new PostCSS-based configuration approach that differs significantly from v3's `tailwind.config.js`. During development, the v4 migration path was found to be incompatible with certain Angular 19 tooling. Rather than downgrading to Tailwind v3 or fighting configuration issues, the decision was made to build a lightweight custom design system using CSS variables, which provides:

- Full theme switching via a single `data-theme` attribute
- No build tooling dependencies
- Complete control over the design language
- Better performance (no utility class bloat)

Light Theme Color Palette

Variable	Value	Usage
`--bg-primary`	`#ffffff`	Main page background
`--bg-secondary`	`#f5f7fa`	Layout areas, inset sections
`--bg-tertiary`	`#eef1f5`	Deeply nested elements
`--bg-sidebar`	`#111318`	Sidebar background (always dark)
`--bg-card`	`#ffffff`	Card surfaces
`--bg-input`	`#f5f7fa`	Input field backgrounds
`--bg-hover`	`#f0f2f5`	Hover state backgrounds
`--text-primary`	`#111827`	Main body text (near-black)
`--text-secondary`	`#6b7280`	Descriptions, metadata
`--text-muted`	`#9ca3af`	Labels, timestamps, placeholders
`--text-sidebar`	`#a1a7b4`	Sidebar text
`--border`	`#e5e7eb`	Card borders, input borders
`--border-light`	`#f0f1f3`	Subtle separators

`--accent`	`#10b981`	Primary accent (emerald green)
`--accent-hover`	`#059669`	Darker green for hover states
`--accent-light`	`#ecfdf5`	Light green for badge backgrounds
`--accent-subtle`	`rgba(16,185,129,0.1)`	Semi-transparent green for chips
`--accent-glow`	`rgba(16,185,129,0.4)`	Green glow for button shadows
`--red`	`#ef4444`	Errors, danger, high priority
`--orange`	`#f59e0b`	Warnings, medium priority
`--green`	`#10b981`	Success, low priority
`--blue`	`#3b82f6`	Info, in-progress status

Dark Theme Color Palette

Applied when `<html data-theme="dark">` :

Variable	Value	Notes
`--bg-primary`	`#09090b`	Near-black, true dark background
`--bg-secondary`	`#111113`	Slightly lighter
`--bg-tertiary`	`#1a1a1e`	Third level
`--bg-sidebar`	`#09090b`	Matches main background (sidebar blends in)
`--bg-card`	`#131316`	Slightly lifted from background
`--bg-input`	`#1a1a1e`	Distinguishable from cards
`--bg-hover`	`#1e1e22`	Subtle hover highlight
`--text-primary`	`#f4f4f5`	Near-white for dark backgrounds
`--text-secondary`	`#a1a1aa`	Medium gray
`--text-muted`	`#71717a`	Darker gray (still readable)
`--border`	`#27272a`	Dark borders
`--accent`	`#34d399`	Lighter green for dark background contrast
`--accent-hover`	`#10b981`	Slightly darker on hover
`--accent-light`	`#022c22`	Very dark green for badges

The dark theme uses a true-black/dark-gray color scheme with zinc-toned grays (`#09090b` , `#131316` , `#1a1a1e`). The accent color shifts from `#10b981` to the lighter `#34d399` for better contrast on dark surfaces. Shadow opacities are increased in dark mode because they need to be visible against dark backgrounds.

Typography

The application uses the Inter font family loaded from Google Fonts with weights 300-800:

```
@import url('https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700;800&display=swap');
```

Base font size is 14px set on `html, body`. Line height is 1.5. Font smoothing is enabled via `-webkit-font-smoothing: antialiased`.

Component Classes

The global stylesheet defines reusable classes used across all components:

Class	Description
`.card`	Bordered, rounded container with shadow. Used for task rows, stat cards, filter bars
`.btn-primary`	Green accent button with hover lift and glow effect. Used for main CTAs
`.btn-ghost`	Transparent button with border. Used for Cancel, Reset, secondary actions
`.btn-danger`	Red button. Used for destructive actions (Stop Timer, Delete)
`.icon-btn`	Small transparent button for icon-only actions (edit, delete, close)
`.input`	Styled text input with green focus ring. Used for all form fields
`select.input`	Dropdown select with custom SVG arrow
`textarea.input`	Text area with vertical resize
`.label`	Small uppercase form label
`.badge`	Pill-shaped indicator base class
`.badge-high`	Red badge for HIGH priority
`.badge-medium`	Amber badge for MEDIUM priority
`.badge-low`	Green badge for LOW priority
`.badge-todo`	Gray badge for TODO status
`.badge-in-progress`	Blue badge for IN_PROGRESS status
`.badge-done`	Green badge for DONE status
`.badge-cancelled`	Red badge for CANCELLED status
`.nav-link`	Sidebar navigation link
`.nav-link.active`	Active nav link with green left-border accent
`.form-group`	Vertical stack (label + input)
`.form-row`	Two-column grid for side-by-side fields
`.form-actions`	Right-aligned button row
`.page`	Vertical flex container for page content
`.page-header`	Space-between layout (title + action buttons)
`.empty`	Centered empty state message
`.animate-in`	fadeIn animation (opacity 0 to 1, translateY 8px to 0)
`.toast-container`	Fixed bottom-right container for toasts
`.toast`	Individual toast notification with slideUp animation

`.toast-success`	Green left border
`.toast-error`	Red left border
`.toast-info`	Blue left border
`.toast-warning`	Amber left border
`.status-select`	Compact inline dropdown for task list quick-status-change

How Theme Switching Works

1. `ThemeService.toggle()` calls `applyTheme('dark')` or `applyTheme('light')`
2. `applyTheme` sets `document.documentElement.setAttribute('data-theme', t)`
3. The CSS selector `[data-theme="dark"]` overrides all `--variable` values
4. Every element using `var(--variable)` instantly reflects the new colors
5. The `transition: background 0.25s ease, color 0.25s ease` on `html, body` creates a smooth visual transition

Animations

Two keyframe animations are defined globally:

```
@keyframes fadeIn {
  from { opacity: 0; transform: translateY(8px); }
  to   { opacity: 1; transform: translateY(0); }
}

@keyframes slideUp {
  from { opacity: 0; transform: translateY(16px); }
  to   { opacity: 1; transform: translateY(0); }
}
```

- `fadeIn` is applied via the `.animate-in` class, used for page content entering the viewport
- `slideUp` is applied to `.toast` elements, creating a bottom-to-top entrance for notifications

6.6 Shared Components

LayoutComponent

`shared/components/layout/layout.component.ts`

The app shell that wraps all authenticated routes. Renders as a flex container with the sidebar on the left and the main content area on the right. The content area includes the topbar at the top and the `<router-outlet>` for page content below it. Also includes the `<app-toast-container>` for notifications.

SidebarComponent

`shared/components/sidebar/sidebar.component.ts`

The left navigation panel that is always visible on authenticated pages. Contains:

- Application logo and name
- Navigation links (Dashboard, My Plans, Goals, Buy List, Calendar) with active-state highlighting via `routerLinkActive`
- User categories section with colored dot indicators
- Category manager trigger (add/edit categories)
- Theme toggle button (sun/moon icon)
- User profile section (avatar, display name, email)
- Logout button

The sidebar has a dark background (`--bg-sidebar: #111318`) in both light and dark themes, providing a consistent look. In dark mode, the sidebar background shifts to `#09090b` to match the main background.

TopbarComponent

`shared/components/topbar/topbar.component.ts`

The horizontal bar at the top of the content area. Displays the current page title, a theme toggle button, and the user's avatar (or initials fallback). The page title is typically set based on the current route.

ModalComponent

`shared/components/modal/modal.component.ts`

A generic dialog wrapper that provides a backdrop overlay and a centered content box. Used by forms (task form, goal form, wishlist form) and the category manager. Supports:

- Click-outside-to-close via backdrop click handler
- Close button in the header
- Customizable title via input property
- Content projection for the modal body

ToastContainerComponent

`shared/components/toast-container/toast-container.component.ts`

Reads the `ToastService.toasts` signal and renders each active toast notification in the bottom-right corner. Each toast displays:

- A colored left border indicating the type (success/error/info/warning)
- The message text
- Click-to-dismiss behavior

Toasts stack vertically with a 0.5rem gap and automatically dismiss after their configured duration.

ConfirmDialogComponent

`shared/components/confirm-dialog/confirm-dialog.component.ts`

A styled confirmation dialog that replaces the browser's native `confirm()` prompt. Used for destructive actions like deleting plans, goals, and wishlist items. Features:

- **Red X icon** -- Visual danger indicator at the top of the dialog
- **Title and message** -- Customizable title (e.g., "Delete Plan") and message that includes the plan name for clarity
- **Action buttons** -- "Cancel" (ghost style) and "Delete" (danger style) buttons
- **Smooth animation** -- Fade-in and scale animation on open
- **Dark overlay backdrop** -- Semi-transparent backdrop that dims the rest of the page
- **Event emitters** -- Emits `confirmed` or `cancelled` events for the parent component to handle

This component provides a consistent, branded confirmation experience across the app instead of relying on browser-native dialogs that cannot be styled.

CategoryManagerComponent

`shared/components/category-manager/category-manager.component.ts`

A popup form component for creating and editing categories. Accessible from the sidebar. Provides:

- Name input field
- Color picker (hex color input)
- Optional icon field
- Save and delete buttons
- Inline editing of existing categories

6.7 Feature Pages

Login / Register Pages

Login (`features/auth/login/login.component.ts`):

- Full-screen layout without sidebar
- Email and password input fields
- Submit button with loading state (disabled while request is in flight)
- Error display for invalid credentials
- Link to register page
- On success: stores tokens, sets current user, navigates to `/dashboard`

Register (`features/auth/register/register.component.ts`):

- Full-screen layout without sidebar
- Email, username, password, and optional display name fields
- Form validation (required fields, email format)
- Submit button with loading state
- Error display for duplicate email/username
- Link to login page
- On success: same as login (stores tokens, navigates to dashboard)

Dashboard (Today) Page

Component: `features/dashboard/dashboard.component.ts`

The landing page after login. Displays:

- **Greeting** -- Time-based greeting ("Good morning," "Good afternoon," etc.) with the user's display name
- **Stats Cards** -- Quick metrics: total tasks, completed today, active tasks, active goals, wishlist items count, wishlist total USD. Each card is a `.card` element with an icon, number, and label.
- **Today's Timeline** -- Tasks due today displayed in a vertical timeline, sorted by start time. Each task shows its type icon, title, time range, and status badge.
- **Goal Progress** -- Active goals with progress bars showing percentage completion. Clicking a goal navigates to the goal detail page.
- **Activity Feed** -- Recent updates from tasks, goals, and wishlist merged into a chronological list.

Data loading: On component init, three parallel API calls are made -- `GET /api/dashboard/stats` , `GET /api/tasks/today` , and `GET /api/dashboard/activity` . Loading states are tracked per-section.

My Plans (Task List) Page

Component: `features/tasks/task-list/task-list.component.ts`

The central task management interface. Displays:

- **Page Header** -- "My Plans" title with "Add Plan" button that opens the task form modal
- **Type Filter Chips** -- Horizontal row of plan type chips (All, Task, Trip, Train, Dinner, Meeting, Event, Reminder). Each chip shows the type icon and count. Clicking a chip filters the list.
- **Status/Priority Filters** -- Additional filter dropdowns for status (All, TODO, IN_PROGRESS, DONE) and priority (All, LOW, MEDIUM, HIGH)
- **Plan Rows** -- Each task is displayed as a row within a `.card` container, showing:
  - Type icon (colored circle with emoji)
  - Title
  - Due date and time range (if set)
  - For **trip** plans: date range display showing start and end dates (e.g., "Jun 8 -> Jun 15")
  - For **train** plans: departure time with clock icon
  - Priority badge
  - Status select dropdown (inline status change without opening detail page)
  - Category tag (if assigned)

    - Delete button

Sorting: Tasks are sorted by date with the earliest due dates first. Tasks without a due date appear at the end of the list.

User interactions:

- Click a plan row to navigate to `/tasks/:id` (detail page)
- Change status via inline dropdown (triggers `PATCH /api/tasks/:id` immediately)
- Click "Add Plan" to open the task form in create mode
- Click delete icon to remove a task (with styled confirm dialog)

Task Detail Page

Component: `features/tasks/task-detail/task-detail.component.ts`

Full information view for a single task/plan. Shows:

- **Header** -- Title, type badge, status badge, priority badge, and Google Calendar sync badge (if synced)
- **Description** -- Full description text (if set)
- **Info Grid** -- A clean grid layout with icons showing all relevant fields for the plan type:
    - Due date, start/end time, location, category
    - Type-specific fields: end date (trips), boarding/destination stations and train number (trains), departure time (trains), meeting link (meetings)
- **Edit Button** -- Opens the task form modal in edit mode for inline editing
- **Delete Button** -- Removes the task with a styled confirm dialog (see Confirm Dialog below)

The timer feature has been removed from the detail page. Time tracking fields (`trackedMins` , `isTimerActive` , `timerStartAt`) remain in the data model but are no longer surfaced in the UI.

Task Form Page

Component: `features/tasks/task-form/task-form.component.ts`

Create or edit a plan. Used as a modal or inline form. Contains:

- **Plan Type Selector** -- Grid of type buttons with icons and colors. Clicking selects the type.
- **Title Input** -- Required field
- **Description Textarea** -- Optional
- **Status Select** -- TODO, IN_PROGRESS, DONE, CANCELLED
- **Category Select** -- Dropdown populated from user's categories
- **Submit Button** -- "Create Plan" or "Update Plan" depending on mode

Dynamic Form Fields Per Plan Type

The form dynamically shows context-specific fields based on the selected plan type. When the user selects a type, only the relevant fields are displayed:

Plan Type	Fields Shown
Task	Date, Priority, Start Time, End Time
Trip	Start Date, End Date, Destination/Location, Priority
Train	Journey Date, Departure Time, Boarding Station, Destination Station, Train Number, Priority
Dinner	Date, Time, Restaurant/Location
Meeting	Date, Start Time, End Time, Location, Meeting Link, Priority
Event	Date, Time, Venue, Priority
Reminder	Date, Time

This approach ensures the form remains clean and contextually relevant. A train plan prompts for boarding station and departure time rather than showing generic start/end time fields. A dinner plan asks for restaurant/location and time without unnecessary priority or end-time inputs.

In **create mode**, the form initializes with defaults (type: task, status: TODO, priority: MEDIUM). In **edit mode**, the form is pre-populated with the existing task data.

Form Reset on Save: After successfully saving a plan (create or update), the form resets all fields to their defaults. This ensures that reopening the form modal always presents a clean, empty form rather than stale data from the previous entry.

Edit Mode Pre-fill Fix: The TaskFormComponent uses both `OnInit` AND `OnChanges` lifecycle hooks to pre-fill the form. `OnInit` alone doesn't work because Angular's `@if` directive in the ModalComponent delays input binding. `OnChanges` detects when the `[task]` input changes and calls `fillForm()` which patches all form fields from the existing task data. When `task` is null (create mode), the form resets to defaults.

Goals List Page

Component: `features/goals/goal-list/goal-list.component.ts`

Displays all goals as cards with:

- **Goal Card** -- Title, description snippet, status badge, target date
- **Progress Bar** -- Visual 0-100% progress bar with percentage label
- **Milestone Count** -- "X of Y milestones completed"
- **CRUD Actions** -- Add Goal button, click to view detail, edit and delete options

Goal Detail Page

Component: `features/goals/goal-detail/goal-detail.component.ts`

The most complex page, showing the full three-level hierarchy:

- **Goal Header** -- Title, description, status, target date, overall progress bar
- **Milestones List** -- Ordered list of milestones, each showing:
  - Title and description
  - Completion checkbox (triggers `completeMilestone` API call)
  - Due date (if set)
  - Expand/collapse to show mini-goals
  - **Nested Mini-Goals** -- Checklist items under each milestone:
    - Checkbox toggle (triggers `toggleMiniGoal` API call)
    - Title
    - Delete button
  - **Add Mini-Goal Form** -- Inline text input to add new mini-goals to a milestone
- **Add Milestone Form** -- Form to add new milestones with title, description, due date, and order
- **Edit/Delete Goal** -- Action buttons at the goal level

Goal Form Page

Component: `features/goals/goal-form/goal-form.component.ts`

Create or edit a goal. Contains:

- Title input (required)
- Description textarea
- Status select (ACTIVE, COMPLETED, PAUSED, ABANDONED)
- Target date picker
- Submit button

Buy List (Wishlist) Page

Component: `features/wishlist/wishlist-list/wishlist-list.component.ts`

Displays wishlist items as product cards in a grid layout:

- **Product Card** -- Each card shows:
  - Product image (if `imageUrl` is set, otherwise a placeholder)
  - Product name
  - Price with currency symbol
  - Priority badge
  - Status badge (WANTED, PURCHASED, ARCHIVED)
  - "View" link (if `url` is set, opens product page in new tab)
  - "Mark as Purchased" button (triggers purchase API call)
  - Edit and delete buttons
  - Category tag (if assigned)
- **Filters** -- Status filter (WANTED, PURCHASED, ARCHIVED) and priority filter

Wishlist Form Page

Component: `features/wishlist/wishlist-form/wishlist-form.component.ts`

Create or edit a wishlist item. Contains:

- Name input (required)
- Description textarea
- Price input (number)
- Currency select (USD, EUR, GBP, INR)
- Product URL input
- Image URL input
- Priority select
- Category select
- Submit button

Calendar Page

Component: `features/calendar/calendar.component.ts`

A monthly calendar view showing tasks plotted on their due dates:

- **Month Navigation** -- Previous/next month arrows with the current month/year display
- **Day Grid** -- 7-column grid showing each day of the month. Days with tasks show colored dots or chips indicating task types.
- **Multi-Day Trip Spanning** -- Trip plans with an `endDate` span across all days in their date range. For example, a trip from Jun 8 to Jun 15 appears on all 8 days in the calendar grid, not just the start date.
- **Weekend Highlighting** -- Saturday and Sunday columns have a visually distinct appearance:
  - Darker background shade to differentiate from weekdays
  - Green-tinted date numbers and day labels
  - Uses CSS variables (`--weekend-bg`) for proper dark mode support, ensuring the weekend colors adapt correctly in both light and dark themes
- **Day Detail Panel** -- Clicking a day opens a side panel showing all tasks for that date with type icons, titles, time ranges, and statuses.
- **Add Task from Calendar** -- The day panel includes an "Add Plan" button that opens the task form with the selected date pre-filled.

Data loading: Calls `GET /api/dashboard/calendar?year=YYYY&month=MM` when the component mounts or when the user navigates to a different month.

---

7. Data Models

User

Field	Type	Description
`id`	`string`	UUID generated at registration
`email`	`string`	Unique email address for login
`username`	`string`	Unique handle for display
`passwordHash`	`string`	bcrypt hash (never sent to client)
`displayName`	`string | null`	Friendly name shown in UI
`avatarUrl`	`string | null`	Profile picture URL (reserved)
`refreshToken`	`string | null`	Current refresh token (never sent to client)
`createdAt`	`string`	ISO timestamp
`updatedAt`	`string`	ISO timestamp

Firebase path: `users/{userId}`

Task (Plan)

Field	Type	Default	Description
`id`	`string`	UUID	Unique identifier
`title`	`string`	(required)	Plan name
`description`	`string | null`	`null`	Detailed description
`type`	`PlanType`	`'task'`	One of: task, trip, train, dinner, meeting, event, reminder
`status`	`TaskStatus`	`'TODO'`	One of: TODO, IN_PROGRESS, DONE, CANCELLED
`priority`	`Priority`	`'MEDIUM'`	One of: LOW, MEDIUM, HIGH
`dueDate`	`string | null`	`null`	ISO date for scheduling
`startTime`	`string | null`	`null`	HH:mm start time
`endTime`	`string | null`	`null`	HH:mm end time
`location`	`string | null`	`null`	Venue or place
`completedAt`	`string | null`	`null`	Set when status becomes DONE
`estimatedMins`	`number | null`	`null`	User's time estimate
`trackedMins`	`number`	`0`	Cumulative timer tracking
`isTimerActive`	`boolean`	`false`	Whether timer is running
`timerStartAt`	`string | null`	`null`	When timer was last started
`userId`	`string`	(from JWT)	Owner reference
`categoryId`	`string | null`	`null`	Optional category reference
`category`	`Category | null`	(populated)	Joined category object
`endDate`	`string | null`	`null`	End date for multi-day plans (e.g., trip end date)

`boardingStation`	`string | null`	`null`	Train boarding station
`destinationStation`	`string | null`	`null`	Train destination station
`trainNumber`	`string | null`	`null`	Train number or name
`departureTime`	`string | null`	`null`	Train departure time (HH:mm)
`meetingLink`	`string | null`	`null`	Meeting video call URL
`googleEventId`	`string | null`	`null`	Google Calendar event ID (set when synced)
`createdAt`	`string`	(auto)	ISO timestamp
`updatedAt`	`string`	(auto)	ISO timestamp

Firebase path: `tasks/{taskId}`

Goal

Field	Type	Default	Description
`id`	`string`	UUID	Unique identifier
`title`	`string`	(required)	Goal name
`description`	`string | null`	`null`	Detailed description
`status`	`GoalStatus`	`'ACTIVE'`	One of: ACTIVE, COMPLETED, PAUSED, ABANDONED
`targetDate`	`string | null`	`null`	Target completion date
`progress`	`number`	`0`	0-100 percentage (auto-calculated)
`userId`	`string`	(from JWT)	Owner reference
`milestones`	`GoalMilestone[]`	(populated)	Attached milestones
`createdAt`	`string`	(auto)	ISO timestamp
`updatedAt`	`string`	(auto)	ISO timestamp

Firebase path: `goals/{goalId}`

GoalMilestone

Field	Type	Default	Description
`id`	`string`	UUID	Unique identifier
`title`	`string`	(required)	Milestone name
`description`	`string | null`	`null`	Details about this checkpoint
`status`	`MilestoneStatus`	`'PENDING'`	PENDING or COMPLETED
`dueDate`	`string | null`	`null`	Optional target date
`completedAt`	`string | null`	`null`	Set when completed
`order`	`number`	`0`	Display sort order
`goalId`	`string`	(from route)	Parent goal reference

`miniGoals`	`MiniGoal[]`	(populated)	Attached mini-goals
`createdAt`	`string`	(auto)	ISO timestamp
`updatedAt`	`string`	(auto)	ISO timestamp

Firebase path: `milestones/{milestoneId}`

MiniGoal

Field	Type	Default	Description
`id`	`string`	UUID	Unique identifier
`title`	`string`	(required)	Small actionable step
`status`	`MilestoneStatus`	`'PENDING'`	PENDING or COMPLETED (toggleable)
`completedAt`	`string | null`	`null`	Set when completed, cleared on uncomplete
`milestoneId`	`string`	(from route)	Parent milestone reference
`goalId`	`string`	(from route)	Grandparent goal reference (denormalized)
`createdAt`	`string`	(auto)	ISO timestamp
`updatedAt`	`string`	(auto)	ISO timestamp

Firebase path: `minigoals/{miniGoalId}`

WishlistItem

Field	Type	Default	Description
`id`	`string`	UUID	Unique identifier
`name`	`string`	(required)	Product name
`description`	`string | null`	`null`	Notes about the product
`price`	`number | null`	`null`	Price for budget tracking
`currency`	`string`	`'USD'`	Currency code (USD, EUR, GBP, INR)
`url`	`string | null`	`null`	Link to product page
`imageUrl`	`string | null`	`null`	Product image URL
`status`	`WishlistItemStatus`	`'WANTED'`	WANTED, PURCHASED, or ARCHIVED
`priority`	`Priority`	`'MEDIUM'`	Reuses task priority levels
`purchasedAt`	`string | null`	`null`	Set when marked as purchased
`userId`	`string`	(from JWT)	Owner reference
`categoryId`	`string | null`	`null`	Optional category reference
`category`	`Category | null`	(populated)	Joined category object
`createdAt`	`string`	(auto)	ISO timestamp
`updatedAt`	`string`	(auto)	ISO timestamp

Firebase path: `wishlist/{itemId}`

Category

Field	Type	Default	Description
`id`	`string`	UUID	Unique identifier
`name`	`string`	(required)	Display label
`color`	`string`	`'#6366f1'`	Hex color for the dot indicator
`icon`	`string | null`	`null`	Optional emoji/icon (reserved)
`userId`	`string`	(from JWT)	Owner reference
`createdAt`	`string`	(auto)	ISO timestamp
`updatedAt`	`string`	(auto)	ISO timestamp

Firebase path: `categories/{categoryId}`

Relationships

```
User  1---*  Task          (userId)
User  1---*  Goal          (userId)
User  1---*  WishlistItem  (userId)
User  1---*  Category      (userId)

Goal       1---*  GoalMilestone  (goalId)
GoalMilestone 1---*  MiniGoal   (milestoneId, goalId)

Category   1---*  Task           (categoryId)
Category   1---*  WishlistItem   (categoryId)
```

---

8. Security

JWT-Based Authentication

All API endpoints (except `/api/auth/register`, `/api/auth/login`, and `/api/auth/refresh`) require a valid JWT access token in the `Authorization: Bearer <token>` header. The token is validated on every request by the globally registered `JwtAuthGuard`.

Global Guard with @Public() Opt-Out

The `APP_GUARD` pattern means security is on by default. Developers must explicitly mark routes as public using the `@Public()` decorator, which attaches `SetMetadata(IS_PUBLIC_KEY, true)` to the route handler. The `JwtAuthGuard` checks for this metadata and skips token validation only when present. This approach prevents accidental exposure of sensitive endpoints.

Ownership Verification

Every mutation operation (create, update, delete) and most read operations verify that the requesting user owns the resource:

```
private async ensureOwnership(userId: string, id: string) {
  const item = await this.firebase.get<any>(`collection/${id}`);
  if (!item) throw new NotFoundException();
  if (item.userId !== userId) throw new ForbiddenException();
```

```
  return item;
}
```

This pattern is implemented consistently in `TasksService`, `GoalsService`, `WishlistService`, and `CategoriesService`. A user can never read, modify, or delete another user's data.

Password Hashing

User passwords are hashed with bcrypt using 12 salt rounds before storage. Plaintext passwords are never stored or logged. During login, `bcrypt.compare()` is used to verify the password against the stored hash.

CORS Configuration

The NestJS application enables CORS with a strict origin policy:

```
app.enableCors({
  origin: 'http://localhost:4200',
  credentials: true,
});
```

Only the Angular frontend running on `localhost:4200` is allowed to make API requests. The `credentials: true` option permits cookies and authorization headers.

Input Validation

The global `ValidationPipe` enforces strict input validation on all incoming requests:

```
app.useGlobalPipes(new ValidationPipe({
  whitelist: true,           // Strip unknown properties
  forbidNonWhitelisted: true, // Reject requests with unknown properties
  transform: true,           // Auto-transform to DTO instances
}));
```

This prevents:

- **Mass assignment attacks** -- Unknown properties are stripped (`whitelist`) or rejected (`forbidNonWhitelisted`)
- **Type confusion** -- Payloads are automatically transformed to the correct DTO types
- **Missing required fields** -- DTO decorators like `@IsString()`, `@IsEmail()` enforce field presence and format

Sensitive Data Exclusion

The `GET /auth/me` endpoint returns the user object from the JWT payload, which contains only `id`, `email`, `username`, `displayName`, and `avatarUrl`. The `passwordHash` and `refreshToken` fields from the database are never included in API responses.

---

9. Theme System

How CSS Custom Properties Work

The theme system is built entirely on CSS custom properties (CSS variables). All color values in the application reference variables like `var(--bg-primary)` and `var(--text-primary)` instead of hardcoded hex values. This means changing the value of a CSS variable instantly updates every element that references it.

Light vs Dark Variable Mapping

The light theme variables are defined on `:root` (the `<html>` element), making them the default. The dark theme variables are defined on `[data-theme="dark"]`, a selector that matches when the `data-theme` attribute is set to `"dark"`. When the attribute is toggled, all CSS variable references automatically resolve to the new values.

Key differences between themes:

Aspect	Light	Dark
Background	White (#ffffff)	Near-black (#09090b)
Text	Dark (#111827)	Light (#f4f4f5)
Cards	White (#ffffff)	Dark gray (#131316)
Accent	Emerald (#10b981)	Lighter emerald (#34d399)
Borders	Light gray (#e5e7eb)	Dark gray (#27272a)
Shadows	Low opacity (0.05-0.1)	Higher opacity (0.3-0.5)
Badges	Light tinted backgrounds	Dark tinted backgrounds

How ThemeService Toggles Themes

```
toggle() {
  this.applyTheme(this.theme() === 'light' ? 'dark' : 'light');
}

private applyTheme(t: Theme) {
  this.theme.set(t);                                        // Update signal
  document.documentElement.setAttribute('data-theme', t);   // Trigger CSS swap
  localStorage.setItem(this.STORAGE_KEY, t);                // Persist choice
}
```

The three-step process ensures:

1. Components reading the `theme` signal re-render with the correct icon
2. CSS variables switch to the new theme's values
3. The user's preference survives page reloads

System Preference Detection

On first visit (no saved preference in localStorage), the service respects the operating system's color scheme preference:

```
const preferred = window.matchMedia('(prefers-color-scheme: dark)').matches ? 'dark' : 'light';
this.applyTheme(saved ?? preferred);
```

If the user has their OS set to dark mode, the app starts in dark mode. If the user manually toggles the theme, that preference is saved to localStorage and takes priority on subsequent visits.

Sidebar-Specific Theme Variables

The sidebar has its own dark background in both themes. In light mode, `--bg-sidebar: #111318` creates a dark sidebar against the white page. In dark mode, `--bg-sidebar: #09090b` makes the sidebar blend with the main background. Sidebar text uses `--text-sidebar` which is a blue-gray tone optimized for readability against the dark sidebar background.

---

10. State Management

Why BehaviorSubject (Not NgRx)

Daily Organizer uses a simple state management approach: Angular signals for local and shared reactive state, and RxJS Observables (via `HttpClient`) for asynchronous data flows. There is no NgRx, NGXS, or other state management library.

This decision was made because:

- **Application complexity does not warrant a store** -- The app has clear data ownership (each feature page manages its own data), and there is no complex state sharing between unrelated components.
- **Services are sufficient** -- Angular services with signals or BehaviorSubjects provide the same capabilities as a store for this use case.
- **Reduced boilerplate** -- NgRx requires actions, reducers, effects, and selectors for every feature. For a personal organizer, this would be over-engineering.

Signal-Based Reactive State

Angular 19 signals are used for state that needs to be reactively read by templates:

- `AuthService.currentUser` -- Read by the sidebar and topbar to display user info. Updated on login/register and cleared on logout.
- `ThemeService.theme` -- Read by toggle buttons to show the correct icon. Updated on toggle.
- `ToastService.toasts` -- Read by the toast container to render notifications. Updated on show/dismiss.

Signals provide synchronous, glitch-free reads without subscription management. Templates that use signals via `service.signal()` automatically re-render when the value changes.

Optimistic Updates in Services

Feature services (task, goal, wishlist, category) follow a fetch-on-demand pattern:

1. Component calls the service method (e.g., `taskService.getAll()`)
2. Service makes an HTTP request and returns an Observable
3. Component subscribes and updates its local state (often a signal or plain variable)
4. For mutations (create, update, delete), the component either:
    - Refetches the list after the mutation succeeds (simple but makes an extra API call)
    - Updates the local array optimistically based on the response

How Components Subscribe

Components typically subscribe to HTTP Observables in one of two ways:

1. **In the component class** -- Using `.subscribe()` in `ngOnInit` or event handlers, storing the result in a local property or signal.
2. **Via the `tap` operator** -- The `AuthService` uses `tap` to perform side effects (storing tokens, updating signals) within the observable pipeline, letting the caller simply subscribe to trigger the flow.

---

11. Key Design Decisions

Why NestJS

NestJS was chosen as the backend framework because it mirrors Angular's architectural patterns (modules, services, dependency injection, decorators), making it natural for an Angular developer to work with both codebases. Both frameworks use TypeScript, decorators, and a modular structure. This symmetry reduces context-switching and allows shared mental models for concepts like services, guards, and interceptors.

Why Firebase Realtime Database

Firebase Realtime Database was selected over SQL databases (PostgreSQL, MySQL) and other NoSQL options (MongoDB, Firestore) for several reasons:

- **Zero schema management** -- No migrations, no table definitions, no ORM configuration. Data is stored as plain JSON, which maps directly to TypeScript objects.
- **Generous free tier** -- The Firebase Spark plan provides sufficient resources for a personal organizer without any cost.

- **Simplicity** -- The Admin SDK's `set()`, `get()`, `update()`, `remove()` methods are straightforward and require minimal boilerplate.
- **Real-time capability** -- While not currently used, Firebase's real-time listeners could be added in the future for live collaboration features.

The `.env` file still references a `DATABASE_URL` with a PostgreSQL connection string, which is a vestige of an earlier design. The application exclusively uses Firebase for all data storage.

Why Standalone Components

Angular 19 recommends standalone components as the default pattern. The Daily Organizer codebase uses standalone components exclusively (no `NgModule` files for features). Benefits include:

- Each component is self-contained with its own imports
- Better tree-shaking eliminates unused code
- Simpler lazy loading (load a component, not a module)
- Reduced boilerplate (no `declarations` array to maintain)

Why Signals

Angular 19 signals provide a simpler alternative to RxJS `BehaviorSubject` for reactive state that does not involve asynchronous streams. In Daily Organizer, signals are used for:

- Current user state (`AuthService`)
- Theme preference (`ThemeService`)
- Toast notifications (`ToastService`)

These are all synchronous state values that need to be reactively read by templates. Signals provide this with less ceremony than Subjects (no `.getValue()`, no `.subscribe()`, no unsubscription).

Why CSS Variables Over Tailwind

Tailwind CSS v4 is installed as a dependency but not used for component styling. The v4 release introduced a new PostCSS-based architecture that broke compatibility with the existing Angular 19 build tooling at the time of development. Rather than fighting configuration issues or downgrading to Tailwind v3, a custom design system was built using CSS custom properties. This approach provides:

- **Native theme switching** -- A single `data-theme` attribute change swaps all colors instantly
- **No build tool dependency** -- CSS variables work in any browser without preprocessing
- **Full design control** -- Every color, shadow, radius, and spacing value is explicitly defined
- **Smaller output** -- No utility class proliferation in the HTML

Why JWT with Refresh Tokens

JWT authentication with a dual-token strategy was chosen for:

- **Stateless authorization** -- The server does not need to look up session data for every request. The JWT contains the user's identity.
- **Separate concerns** -- The short-lived access token (15 minutes) limits the window of exposure if compromised. The long-lived refresh token (7 days) provides a smooth user experience without frequent re-authentication.
- **Token rotation** -- Each refresh generates a new refresh token, invalidating the old one. This limits the damage from a leaked refresh token.
- **Frontend simplicity** -- Tokens are stored in localStorage and automatically attached to requests by the interceptor.

Why Functional Guards and Interceptors

Angular 19 deprecates class-based guards and interceptors in favor of functional alternatives. The Daily Organizer uses functional guards (`authGuard`, `noAuthGuard`) and a functional interceptor (`authInterceptor`). Benefits:

- Less boilerplate (a function vs. a class with `implements CanActivate`)
- Uses `inject()` for dependency injection inside the function body
- Aligns with Angular's direction for future versions

Multi-Type Task System

Rather than creating separate collections and APIs for tasks, trips, trains, dinners, meetings, events, and reminders, all plan types share a single `tasks` collection with a `type` discriminator field. This was a deliberate choice:

- **Unified filtering** -- All plans appear in one list that can be filtered by type
- **Shared calendar** -- All plan types appear on the calendar without merging separate data sources
- **Simpler API** -- One set of CRUD endpoints handles all types
- **Consistent timer** -- The timer feature works identically for all plan types
- **Common fields** -- All types benefit from priority, status, due date, category, and time fields

---

12. Setup & Running Guide

Prerequisites

Requirement	Version	Notes
Node.js	18+ (LTS recommended)	Download from https://nodejs.org
npm	9+	Ships with Node.js
Angular CLI	19.x	Install globally: `npm install -g @angular/cli`
NestJS CLI	11.x	Install globally: `npm install -g @nestjs/cli` (optional)

Firebase Setup

1. Go to Firebase Console and create a new project (or use an existing one)
2. Navigate to **Project Settings > Service Accounts**
3. Click **Generate New Private Key** to download the service account JSON file
4. Rename the file to `serviceAccountKey.json` and place it in the `backend/` directory root
5. Enable the **Realtime Database** in the Firebase console (choose the region closest to you)
6. Note the database URL (it will be in the format `https://{project-id}-default-rtdb.firebaseio.com`)
7. Update the `databaseURL` in `backend/src/prisma/firebase.service.ts` if your project URL differs

Environment Configuration

The backend uses a `.env` file in the `backend/` directory with the following variables:

```
# Database URL (vestigial -- Firebase is used instead)
DATABASE_URL="postgresql://..."

# JWT access token configuration
JWT_SECRET="your-super-secret-jwt-key-change-in-production-min-32-chars"
JWT_EXPIRES_IN="15m"

# JWT refresh token configuration (must use a DIFFERENT secret)
JWT_REFRESH_SECRET="your-super-secret-refresh-key-change-in-production-min-32-chars"
JWT_REFRESH_EXPIRES_IN="7d"

# Server port
PORT=3000

# Google Calendar Integration (OAuth 2.0)
GOOGLE_CLIENT_ID="your-google-client-id.apps.googleusercontent.com"
GOOGLE_CLIENT_SECRET="your-google-client-secret"
GOOGLE_REDIRECT_URI="http://localhost:3000/api/google/callback"
```

For production, generate strong random secrets (minimum 32 characters) for both `JWT_SECRET` and `JWT_REFRESH_SECRET`. They must be different from each other.

The Google Calendar environment variables are required only if you want to enable Google Calendar integration. See [Section 13: Google Calendar Integration] for setup instructions.

Backend Setup

```
# Navigate to the backend directory
cd backend

# Install dependencies
npm install

# Ensure serviceAccountKey.json is in place
ls serviceAccountKey.json

# Start the development server (with hot reload)
npm run start:dev

# The API will be available at http://localhost:3000/api
```

Available Scripts:

Script	Command	Description
`start`	`nest start`	Start in production mode
`start:dev`	`nest start --watch`	Start with hot reload
`start:debug`	`nest start --debug --watch`	Start with debugger
`start:prod`	`node dist/main`	Run compiled production build
`build`	`nest build`	Compile TypeScript to JavaScript
`lint`	`eslint ... --fix`	Lint and auto-fix TypeScript files
`test`	`jest`	Run unit tests
`test:watch`	`jest --watch`	Run tests in watch mode
`test:cov`	`jest --coverage`	Run tests with coverage report

Frontend Setup

```
# Navigate to the frontend directory
cd frontend

# Install dependencies
npm install

# Start the development server
ng serve
# or
npm start
```

```
# The app will be available at http://localhost:4200
```

Available Scripts:

Script	Command	Description
`start`	`ng serve`	Start dev server on port 4200
`build`	`ng build`	Build for production
`watch`	`ng build --watch --configuration development`	Build with file watching
`test`	`ng test`	Run unit tests via Karma

URLs

Service	URL	Description
Frontend	`http://localhost:4200`	Angular SPA
Backend API	`http://localhost:3000/api`	NestJS REST API
Firebase Console	`https://console.firebase.google.com`	Database management

Quick Start Checklist

1. Install Node.js 18+
2. Clone the repository
3. Place `serviceAccountKey.json` in `backend/`
4. Configure `backend/.env` with JWT secrets
5. Run `cd backend && npm install && npm run start:dev`
6. Run `cd frontend && npm install && ng serve` (in a separate terminal)
7. Open `http://localhost:4200` in your browser
8. Register a new account and start organizing

---

13. Google Calendar Integration

Overview

Tasks and plans with a `dueDate` automatically sync to the user's Google Calendar. This provides a seamless bridge between the Daily Organizer app and the user's existing calendar workflow, ensuring all scheduled plans appear alongside other calendar events.

OAuth 2.0 Flow

The integration uses Google's OAuth 2.0 authorization code flow with the `calendar.events` scope:

1. User clicks **"Connect Google Calendar"** in the sidebar
2. Backend generates a Google OAuth consent URL with the `calendar.events` scope
3. User grants permission on Google's consent screen
4. Google redirects to `/api/google/callback` with an authorization code
5. Backend exchanges the code for tokens (access token + refresh token)
6. Tokens are stored in Firebase under `users/{userId}` with the following fields:
   - `googleAccessToken` -- Short-lived access token for API calls
   - `googleRefreshToken` -- Long-lived refresh token for obtaining new access tokens
   - `googleTokenExpiry` -- Expiration timestamp of the current access token
   - `googleCalendarConnected` -- Boolean flag indicating connection status

7. User is redirected to the dashboard with `?gcal=connected` query parameter
8. Dashboard shows a success toast notification confirming the connection

Auto-Sync Logic

Google Calendar events are automatically managed as part of task CRUD operations:

- **On task CREATE with dueDate** -- Creates a corresponding Google Calendar event and stores the `googleEventId` on the task record
- **On task UPDATE** -- Updates the existing Google Calendar event. If a dueDate is added, creates a new event. If a dueDate is removed, deletes the existing event.
- **On task DELETE** -- Deletes the associated Google Calendar event

All sync operations are **best-effort**: they are wrapped in try/catch blocks and never fail the underlying task operation. If Google Calendar is unreachable or the user's token has been revoked, the task operation still succeeds.

Sync All

`POST /api/google/sync-all` pushes all existing tasks that have a `dueDate` but no `googleEventId` to Google Calendar. This is useful for syncing tasks created before the Google Calendar integration was connected. The endpoint returns `{ synced: number }` indicating how many tasks were synced.

Sidebar Controls: When Google Calendar is connected, the sidebar shows:

- A **"Sync All"** button to trigger bulk sync of unsynced tasks
- A **disconnect X button** to remove the Google Calendar connection

Task-to-Event Mapping

Task Field	Google Calendar Event Field	Notes
`title`	`summary`	Direct mapping
`description`	`description`	Prefixed with `[TYPE]` (e.g., `[MEETING] Team standup`)
`location`	`location`	Direct mapping
`dueDate` + `startTime` + `endTime`	`start.dateTime` / `end.dateTime`	RFC3339 format, `Asia/Kolkata` timezone
`dueDate` only (no time)	`start.date` / `end.date`	All-day event
`type`	`colorId`	Mapped as follows: task=Blueberry, trip=Sage, train=Banana, dinner=Flamingo, meeting=Grape, event=Peacock, reminder=Tangerine

Token Refresh

The `googleapis` OAuth2Client automatically handles expired access tokens. When a token expires, the client uses the stored refresh token to obtain a new access token. New tokens are persisted to Firebase via the `tokens` event listener on the OAuth2Client, ensuring the user never needs to re-authenticate unless they explicitly revoke access.

New Files

File	Description
`backend/src/google-calendar/google-calendar.module.ts`	NestJS module registering the Google Calendar service and controller
`backend/src/google-calendar/google-calendar.service.ts`	OAuth flow, Calendar API CRUD operations, and syncAll logic

`backend/src/google-calendar/google-calendar.controller.ts`	Endpoints: GET /auth, GET /callback, GET /status, POST /disconnect, POST /sync-all
`frontend/src/app/core/services/google-calendar.service.ts`	Frontend service for connect, disconnect, checkStatus, and syncAll API calls

Modified Files

File	Change
`backend/src/tasks/tasks.service.ts`	GoogleCalendarService injected; sync calls added in create/update/remove methods
`backend/src/tasks/tasks.module.ts`	Imports GoogleCalendarModule
`backend/src/app.module.ts`	Imports GoogleCalendarModule
`frontend/src/app/core/models/task.model.ts`	Added `googleEventId: string | null` field
`frontend/src/app/shared/components/sidebar/sidebar.component.ts`	Added Connect/Sync All/Disconnect buttons for Google Calendar
`frontend/src/app/features/dashboard/dashboard.component.ts`	Handles `?gcal=connected` redirect and shows success toast

Prerequisites

To enable Google Calendar integration, you need to configure a Google Cloud project:

1. Go to Google Cloud Console
2. Create a new project (or select an existing one)
3. Navigate to **APIs & Services > Library** and enable the **Google Calendar API**
4. Navigate to **APIs & Services > Credentials** and create an **OAuth 2.0 Client ID** (Web application)
5. Add the redirect URI: `http://localhost:3000/api/google/callback` (or your production URL)
6. Under **OAuth consent screen**, configure the app name, scopes (`calendar.events`), and add test users (required while the app is in "Testing" status)
7. Copy the **Client ID** and **Client Secret** into your `.env` file

Environment Variables

Variable	Description	Example
`GOOGLE_CLIENT_ID`	OAuth 2.0 client ID from Google Cloud Console	`123456789.apps.googleusercontent.com`
`GOOGLE_CLIENT_SECRET`	OAuth 2.0 client secret	`GOCSPX-xxxxxxxxxxx`
`GOOGLE_REDIRECT_URI`	Callback URL registered in Google Cloud Console	`http://localhost:3000/api/google/callback`

---

14. Auto Status Updates

Overview

Tasks and plans automatically transition between statuses based on their scheduled dates and times. This eliminates the need for users to manually update the status of time-bound plans, keeping the dashboard and task list accurate in real time.

Status Transitions

Transition	Trigger
TODO -> IN_PROGRESS	When the plan's start date/time arrives

IN_PROGRESS -> DONE	When the plan's end date/time passes

These transitions are automatic and do not overwrite manually set statuses like CANCELLED.

Type-Specific Logic

Each plan type has its own rules for determining when a plan starts and ends:

Plan Type	Starts At	Ends At
Task	Start time on due date	End time on due date, or end of day if no end time
Trip	Beginning of start date (midnight)	End of endDate (23:59)
Train	Departure time on journey date	30 minutes after departure time
Dinner	Scheduled time on date	2 hours after scheduled time
Meeting	Start time on date	End time on date
Event	Scheduled time on date	3 hours after scheduled time
Reminder	Scheduled time on date	Triggers immediately at set time

Implementation

The auto-status logic runs when tasks are fetched, comparing the current timestamp against each plan's computed start and end times. Plans that have passed their start time but not their end time are moved to IN_PROGRESS. Plans that have passed their end time are moved to DONE. The updated statuses are persisted to Firebase so they remain consistent across sessions.

---

15. Future Scope & Enhancements

Push Notifications / Reminders

Currently, the "reminder" plan type has no actual notification mechanism. A future enhancement could integrate Web Push Notifications to alert users when a reminder's due time arrives. This would involve:

- A service worker for background push handling
- A notification permission flow in the frontend
- A scheduled job on the backend (or Firebase Cloud Functions) that checks for upcoming reminders and sends push messages

Drag-and-Drop Reordering

Milestones within goals currently use a numeric `order` field, but there is no drag-and-drop UI to reorder them. Adding Angular CDK's `DragDropModule` would enable intuitive reordering of milestones, and potentially tasks within a day's timeline.

File Attachments

Tasks and goals could benefit from file attachments (documents, images, screenshots). Firebase Storage could be used to host files, with references stored in the task/goal records.

Collaborative Sharing

The current architecture is single-user (each resource has one `userId`). Future enhancements could add sharing capabilities, allowing users to collaborate on tasks or goals by adding a `sharedWith` array of user IDs and adjusting the ownership verification logic.

Mobile Responsive Design

Implemented. See Section 16 -- Mobile Responsive Design for full details.

PWA Support

Adding a service worker and a web app manifest would enable the application to work as a Progressive Web App, allowing offline access to cached data and installation on mobile home screens.

Data Export (CSV/PDF)

Users may want to export their tasks, goals, or wishlist items for backup or reporting. Adding export endpoints that generate CSV or PDF files would provide this capability.

Recurring Tasks

The current system handles one-time plans. A recurring task feature would allow users to define tasks that repeat daily, weekly, monthly, or on custom schedules. This would require a recurrence pattern field and a scheduled job to generate instances.

Calendar Week View

The current calendar shows a monthly grid. A week view would provide a more detailed, hour-by-hour layout for users who plan their days in detail. This is particularly useful for plan types with start and end times (meetings, trains, events).

Firebase Authentication (Replace Custom JWT)

The current implementation uses custom JWT authentication with passwords stored in Firebase Realtime Database. An alternative approach would be to use Firebase Authentication as the identity provider, which would offer:

- Built-in email/password auth with email verification
- Social login providers (Google, GitHub, etc.)
- Password reset flows
- Reduced backend complexity (no password hashing, no token generation)

However, this would introduce a dependency on Firebase's authentication service and reduce control over the auth flow.

---

16. Mobile Responsive Design

Overview

The application is fully responsive with a mobile-first breakpoint at **768px**. All feature pages adapt their layout, typography, and spacing for smaller screens, ensuring a usable experience on phones and tablets.

Collapsible Sidebar

On screens narrower than 768px, the desktop sidebar is hidden by default. A **mobile top bar** is displayed instead, containing:

- **Hamburger menu button** (left) -- Toggles the sidebar as a slide-in overlay
- **"Daily Organizer" title** (center) -- App branding
- **Dark/Light theme toggle** (right) -- Quick access to theme switching

When the sidebar overlay is open, clicking any navigation link automatically closes it, providing a smooth single-action navigation experience.

Safe Area Support

The sidebar bottom uses `env(safe-area-inset-bottom)` to account for the iPhone notch and home indicator area, ensuring no content is obscured on modern iOS devices.

Per-Page Responsive Behavior

Page	Mobile Adaptation
Dashboard	Stats displayed in a 2x2 grid; panels stack vertically; smaller font sizes

Task List	Plan rows wrap content; filters stack vertically; smaller type chips
Calendar	Compact cells (70px height); full-width day detail panel; time prefix hidden on chips
Goals	Single-column card grid instead of multi-column
Auth Pages	Reduced padding on small screens
Forms	Two-column field rows collapse to single column
Toasts	Full width, positioned at the bottom of the screen

File Changes

File	Change
`frontend/src/app/shared/components/sidebar/layout.component.ts`	Added hamburger menu button, mobile top bar, and overlay toggle logic
`frontend/src/styles.css`	Added responsive utility classes and global media queries
All feature components	Added `@media (max-width: 768px)` queries for page-specific layout adjustments

---

17. Loading Screen for Cold Start

Overview

The dashboard displays a **loading spinner** with a "Loading your plans..." message during the initial data fetch. This addresses the user experience on the Render free tier, where the backend may take up to 50 seconds to respond after being idle.

Behavior

- A hint message is shown below the spinner: "*First load may take ~30s if the server was sleeping*"
- **Auto-retry**: If the first API call fails (e.g., due to a cold-starting server), the dashboard automatically retries after **3 seconds**
- Dashboard content (stats, today's tasks, goal summaries) remains hidden until all three API calls complete successfully

Render Free Tier Note

The backend is deployed on Render's free tier, which has the following characteristics:

Behavior	Detail
Spin-down	Service spins down after 15 minutes of inactivity
Cold start	First request after idle takes approximately 50 seconds
Data safety	All data is stored in Firebase, not on the Render instance -- no data is lost during spin-down
Keep-alive recommendation	Use UptimeRobot (free tier) to ping the backend health endpoint every 14 minutes to prevent spin-down

---

18. Clickable Dashboard Stat Cards

Overview

All four dashboard stat cards are now interactive links that navigate the user to the relevant feature page.

Card Routing

Stat Card	Navigates To
Active Plans	`/tasks`
Done Today	`/tasks`
Goals	`/goals`
To Buy	`/wishlist`

Hover Effect

Each card responds to hover with:

- A **green border** highlight
- A slight **lift** effect (`translateY(-2px)`)
- A subtle **box shadow** for depth

The cards are rendered as `<a>` elements with `routerLink`, ensuring proper accessibility and native link behavior (right-click to open in new tab, etc.).

---

19. Production Deployment

Environment File Replacement

The `angular.json` build configuration includes a `fileReplacements` entry for production builds. This ensures that the production environment file (containing the Render backend URL) is used instead of the development environment file (which points to `localhost:3000`).

Without this configuration, the production build would attempt to call `localhost:3000` for all API requests, causing authentication and all API calls to fail on deployed/mobile clients.

File Changed

File	Change
`angular.json`	Added `fileReplacements` under `configurations.production` to swap the environment file for production builds

---

20. Form Reset Fix

Overview

After saving a plan (create or edit), the form now resets all fields to their default values. Previously, stale data from the last edited plan could persist in the form when it was reopened, leading to confusion and potential data errors. The fix ensures a clean form state every time the user opens the plan creation or editing dialog.

This documentation was generated from the actual codebase of the Daily Organizer application. All version numbers, file paths, code examples, and architectural details reflect the current state of the project.